

Group Report 1

Authors: Shaalin Jaiswal, Pooja Pillai, Likitha Pogula, Vijay Jonnakuti, Daksh Desai, Ethan Nguyen

Course: HON 498 - Barrett College Fellows

PI: Dr. Neha Joshi

Date: October 7, 2026

Part One: GETTING TO KNOW GRAPHS

1: Defining a Graph

A graph is a mathematical structure $G=(V,E)$ where V is the set of vertices or nodes, and E is the set of edges or links connecting pairs of vertices (Ajorlou, 2018). Graphs represent objects and the relationships between them. An example of this is in a student collaboration network, each vertex represents a student, and an edge connects two students who have worked on a project together.

A graph is usually drawn using points for vertices and lines for edges. However, the connections matter more than the positions of the points. Moving the points without changing their connections still represents the same graph. In graph theory, a graph describes a network of relationships. This is different than a graph of a function which shows how one numerical variable changes with another.

When describing a network, we also need to specify which kinds of connections are allowed. A simple graph has no self-loops, which are edges connecting a vertex to itself, and no multiple edges between the same pair of vertices. More general graphs can allow these features, depending on the relationships being modeled (Ajorlou, 2018).

2: Vertices and Edges

To describe the connections precisely, graph theory uses the terms adjacency and incidence. Two vertices are adjacent(or neighbors) when an edge joins them. An edge is incident to each vertex it connects, and these vertices are called its endpoints. Adjacency describes a relationship between two vertices, while incidence describes a relationship between an edge and vertex (Ajorlou, 2018).

The neighborhood of a vertex contains all the vertices directly connected to it by an edge. For example, if B is connected to A and C , its neighborhood is $\{A,C\}$. Two edges can also

share an endpoint. The edges {A,B} and {B,C} both connect to B, so B is their shared endpoint (Ajourlou, 2018).

In the vertex set $V = \{A,B,C,D\}$ and the edge set $E = \{\{A,B\},\{B,C\}\}$. This graph has four vertices and two edges. The edge {A,B} has endpoints A and B, while the {B,C} has endpoints B and C. Also, Vertex D belongs to the graph even though no edge connects it to another vertex, so it is a isolated vertex.

```
In [1]: import networkx as nx
import matplotlib.pyplot as plt

G = nx.Graph()
G.add_nodes_from(["A", "B", "C", "D"])
G.add_edges_from([("A", "B"), ("B", "C")])

positions = {"A": (0,0), "B": (1,0), "C":(2,0), "D": (3.5,0)}
plt.figure(figsize = (8,3))
nx.draw_networkx(G, pos = positions, node_color = "lightblue", node_size = 1200,
plt.title("Vertices and Edges")
plt.margins(0.2)
plt.axis("off")
plt.show()
```

Vertices and Edges



Figure 1: A graph with four vertices and two edges. The two edges share endpoint B, while D is isolated.

From the figure, the neighborhood of B is {A,C}, while the neighborhood of A is {B}. Vertex D has an empty neighborhood since it has no incident edges. D still belongs to the vertex set, showing that a vertex can exist in a graph without having any connections.

3: Directed and Undirected graphs

An undirected graph has edges without direction. An edge connecting A and B represents the same connections from either endpoint is drawn as a line without an arrow (Sedgewick &

Wayne, n.d.). An example of this is a collaboration between two students that can be modeled as an undirected relationship because both students participate in the same connection.

A directed graph which is also called a digraph, has edges with a specific direction. These edges are drawn as arrows. An arrow from A to B represents a connection from A to B, but it doesn't automatically mean that there is a connection from B to A. Also, A can point to B, and B can point to A, but they are two separate directed edges (Nykamp, n.d.). Links between web pages can illustrate why direction matters. One page can link to another without receiving a link back. A directed graph shows which page the link comes from and which page it points to (Nykamp, n.d.).

To compare the two types, consider the vertices A, B, and C arranged in a triangle. In the undirected example, edges connect A to B, B to C, and C to A without arrows. While in the directed example, arrows point from A to B, B to C, and C to A.

```
In [2]: import networkx as nx
import matplotlib.pyplot as plt

undirected = nx.Graph()
undirected.add_edges_from([("A", "B"), ("B", "C"), ("C", "A")])

directed = nx.DiGraph()
directed.add_edges_from([("A", "B"), ("B", "C"), ("C", "A")])

positions = {"A": (0,1), "B": (-1,-0.5), "C": (1,-0.5)}
fig, axes = plt.subplots(1, 2, figsize = (10, 3))

nx.draw_networkx( undirected, pos = positions, ax = axes[0], node_color = "lightblue",
axes[0].set_title("Undirected Graph")

nx.draw_networkx(directed, pos = positions, ax = axes[1], node_color = "lightblue",
axes[1].set_title("Directed Graph")

for ax in axes:
    ax.margins(0.2)
    ax.axis("off")

plt.tight_layout()
plt.show()
```

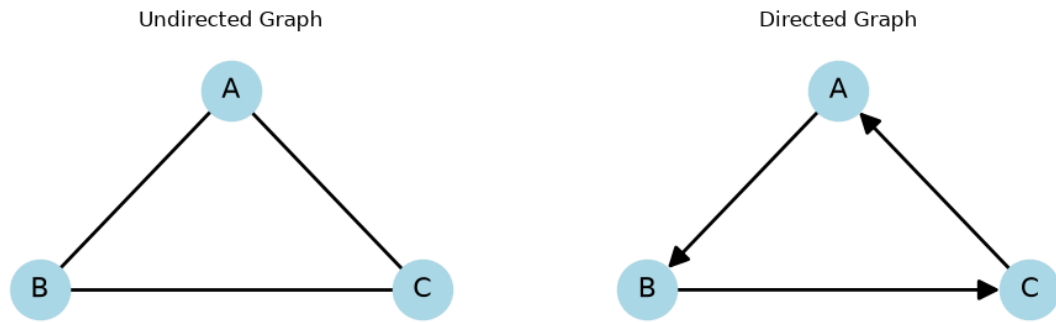


Figure 2: Undirected and directed graphs with three vertices and three edges each. The undirected graph has no arrows, while the directed graph has arrows from A to B, B to C, and C to A.

Both graphs connect the same pairs of vertices, but direction changes how the connections can be followed. In the undirected graph, the edge between A and C can be followed either way. However in the directed graph, the edge points only from C to A. To reach C from A while following the arrows, we must go through B.

4: Weighted and Unweighted graphs

An unweighted graph records whether a connection exists without assigning a numerical weight to each edge. A weighted graph adds a number to each edge to describe something about that connection. Depending on the model, weights might represent distance, travel time, cost, or the strength of an interaction (Ajorlou, 2018; Nykamp, n.d.).

Each weight needs a clear meaning. A larger number could indicate a longer journey in a transportation network or a stronger interaction in a social network. Therefore, larger weights do not always mean better connections.

For an example, consider three locations labeled A, B, and C. All three pairs have a direct route. In the unweighted graph, the edges show only the routes. In the weighted graph, suppose the travel times are 4 minutes between A and B, 3 minutes between B and C, and 10 minutes between A and C.

```
In [3]: import networkx as nx
import matplotlib.pyplot as plt

unweighted = nx.Graph()
unweighted.add_edges_from([("A", "B"), ("B", "C"), ("C", "A")])

weighted = nx.Graph()
weighted.add_weighted_edges_from([("A", "B", 4), ("B", "C", 3), ("A", "C", 10)])

positions = {"A": (0,1), "B": (-1,-0.5), "C": (1,-0.5)}
fig, axes = plt.subplots(1, 2, figsize = (10, 4))

for graph, ax in zip([unweighted, weighted], axes):
```

```

nx.draw_networkx(graph, pos = positions, ax = ax, node_color = "lightblue", nod
ax.margins(0.2)
ax.axis("off")
axes[0].set_title("Unweighted Graph")
axes[1].set_title("Weighted Graph")

edge_labels = {(u,v) : f"{data['weight']} min"
                for u, v, data in weighted.edges(data = True)}

nx.draw_networkx_edge_labels(weighted, pos= positions, edge_labels = edge_labels, a

plt.tight_layout()
plt.show()

```

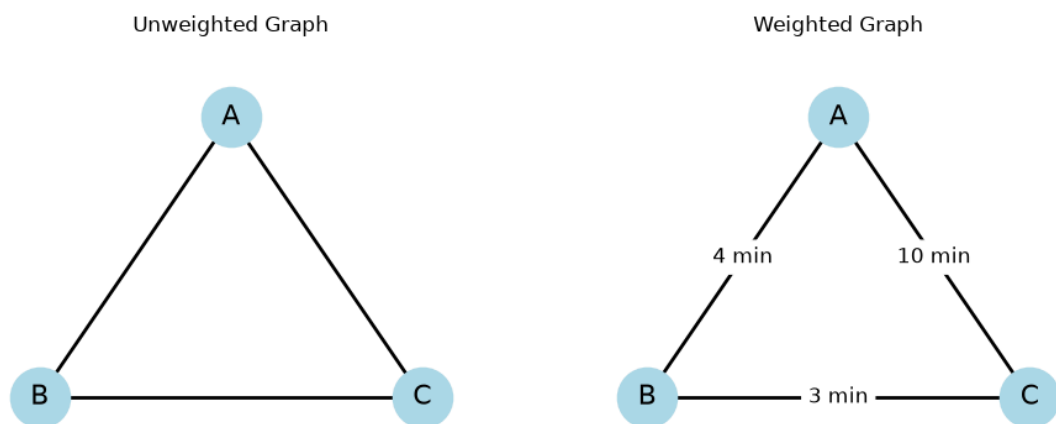


Figure 3: Unweighted and weighted graphs with the same three vertices and edges. In the weighted graph, the edge labels represent travel times of 4, 3, 10 minutes.

In the weighted example, traveling directly from A to C takes 10 minutes, while traveling through B takes 7 minutes(4+ 3). Although the route through B uses more edges, it takes less time. This shows how weights provide information that the unweighted graph does not record.

Direction and weight are separate features. Both graphs in Figure 3 are undirected, but directed graphs can also have weights, such as one-way roads labeled with travel times.

5: Degrees

5.1: Definition of a Degree

The degree of a vertex in a graph is the number of edges incident to it. The word incident in the graph theory context describes the connection between a node (vertex) and an edge (the connecting line). Below is an example of a graph with its degrees notated. As you can

see, the vertex A has a degree of 2 because it has 2 connections to other edges. Similarly, vertex E has a degree of 4 because it has 4 connections to other edges.

```
In [ ]: import matplotlib.pyplot as plt
import networkx as nx

# 1. Create an undirected graph
G = nx.Graph()

# 2. Add edges to match the example described in your text
# Node 'A' is the far-left node (Degree = 2)
# Node '3' is the central node (Degree = 4)
edges = [
    ("A", "E"),
    ("A", "B"),
    ("E", "B"),
    ("E", "C"),
    ("E", "D"),
]
G.add_edges_from(edges)

# 3. Calculate degrees for every node
degrees = dict(G.degree())

# 4. Set fixed positions for a clean layout (Far-left node on the left, node '3' ce
pos = {
    "A": (-2, 0), # Far-left node (Degree 2)
    "E": (0, 0), # Central node (Degree 4)
    "B": (-1, 1), # Top node connecting A and 3
    "C": (1.5, 1), # Top-right neighbor
    "D": (1.5, -1), # Bottom-right neighbor
}

# 5. Create labels showing both the node name/value and its degree
node_labels = {node: f"{node}\ndeg({deg})" for node, deg in degrees.items()}

# 6. Plot the graph
plt.figure(figsize=(9, 5))

# Draw nodes
nx.draw_networkx_nodes(
    G, pos, node_size=2500, node_color="#4B9CD3", edgecolors="black", linewidths=1.
)

# Draw edges
nx.draw_networkx_edges(G, pos, width=2, edge_color="#555555")

# Draw custom labels inside the nodes
nx.draw_networkx_labels(
    G, pos, labels=node_labels, font_size=9, font_color="white", font_weight="bold"
)

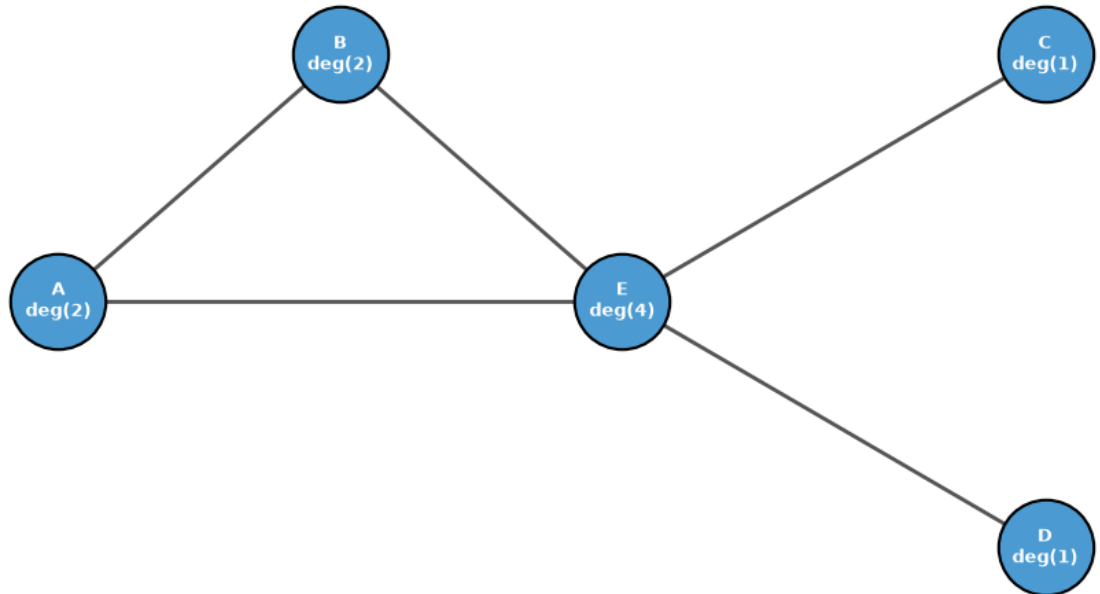
# Customize title and display grid settings
plt.title(
    "Undirected Graph: Node Degree Example",
```

```

    fontsize=14,
    fontweight="bold",
    pad=15,
)
plt.axis("off") # Turn off standard x-y coordinate axes
plt.tight_layout()
plt.show()

```

Undirected Graph: Node Degree Example



5.2: Degree Notation

In the case of a simple graph, a degree can be formally written as $\deg(u)$, where u is a vertex of a graph. Using our previous graph and vertices, vertex A can be denoted as $\deg(2)$, and vertex E can be denoted as $\deg(4)$. Directed graphs have a similar notation but vary slightly due to the direction of edges, which can further be classified as in-degree and out-degree. The in-degree of a vertex is the number of edges where the edge is pointed towards the vertex, also known as a terminal vertex. The out-degree of a vertex is the number of edges where the edge is pointed away from the vertex, also known as the initial vertex. Directed graphs have comma-paired values to represent first the in-degrees and then the out-degrees. Equation-wise, in-degree is often denoted as $\deg^-(u)$ and out-degree is often denoted as $\deg^+(u)$, where u is the vertex. Let us look at an example down below.

```

In [ ]: import matplotlib.pyplot as plt
import networkx as nx

# 1. Create a directed graph (DiGraph)
G = nx.DiGraph()

# 2. Add 8 vertices labelled A through H
nodes = ["A", "B", "C", "D", "E", "F", "G", "H"]
G.add_nodes_from(nodes)

# 3. Add directed edges to create a balanced 8-vertex network

```

```

edges = [
    ("A", "B"),
    ("A", "C"),
    ("B", "D"),
    ("B", "E"),
    ("C", "E"),
    ("C", "F"),
    ("D", "G"),
    ("E", "D"),
    ("E", "G"),
    ("E", "H"),
    ("F", "E"),
    ("F", "H"),
    ("G", "H"),
]
G.add_edges_from(edges)

# 4. Calculate in-degree (deg-) and out-degree (deg+) for every node
in_degrees = dict(G.in_degree())
out_degrees = dict(G.out_degree())

# 5. Set positions for an 8-node layout across 4 columns
pos = {
    "A": (-3.0, 0.0), # Column 1 (Source)
    "B": (-1.0, 1.2), # Column 2 (Top)
    "C": (-1.0, -1.2), # Column 2 (Bottom)
    "D": (1.0, 1.2), # Column 3 (Top)
    "E": (0.0, 0.0), # Column 3 (Center hub)
    "F": (1.0, -1.2), # Column 3 (Bottom)
    "G": (2.5, 0.8), # Column 4 (Top-Right)
    "H": (2.5, -0.8), # Column 4 (Bottom-Right)
}

plt.figure(figsize=(11, 6))

# 6. Draw blue node circles
nx.draw_networkx_nodes(
    G, pos, node_size=1800, node_color="blue", edgecolors="black", linewidths=1.0
)

# 7. Draw straight, crisp directed arrows
nx.draw_networkx_edges(
    G,
    pos,
    width=1.5,
    edge_color="#333333",
    arrowstyle="->",
    arrowsize=18,
    node_size=1800,
    connectionstyle="arc3,rad=0.0", # Straight lines
)

# 8. Draw Letter Labels inside the nodes
node_labels = {node: node for node in G.nodes()}
nx.draw_networkx_labels(
    G, pos, labels=node_labels, font_size=14, font_color="white", font_weight="bold"
)

```

```

)

# 9. Add (In, Out) degree notation above each node (Red for In, Blue for Out)
for node, (x, y) in pos.items():
    in_deg = in_degrees[node]
    out_deg = out_degrees[node]

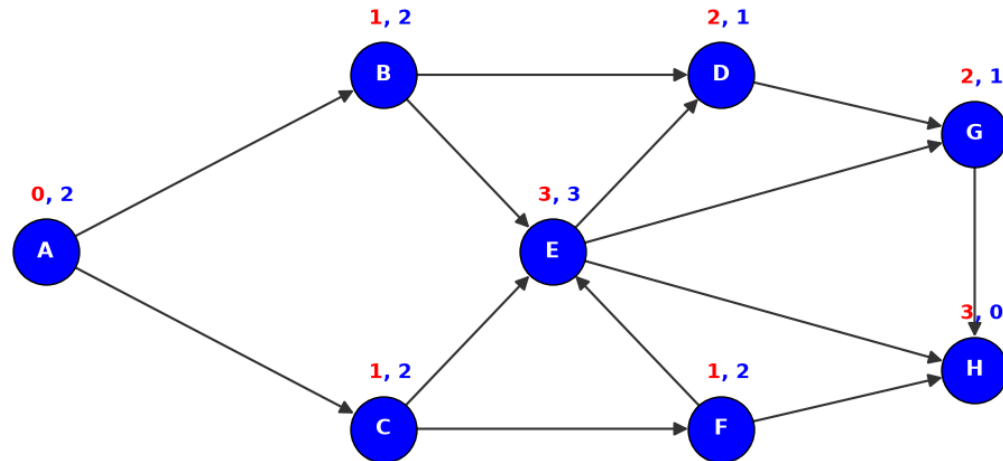
    # Vertical position offset above node
    y_offset = 0.38

    # Red for In-degree
    plt.text(
        x,
        y + y_offset,
        f"{in_deg}",
        color="red",
        fontsize=13,
        fontweight="bold",
        ha="right",
        va="center",
    )
    # Blue for Out-degree
    plt.text(
        x,
        y + y_offset,
        f"{out_deg}",
        color="blue",
        fontsize=13,
        fontweight="bold",
        ha="left",
        va="center",
    )

# Formatting adjustments
plt.xlim(-3.8, 3.3)
plt.ylim(-2.0, 2.0)
plt.title(
    "8-Vertex Directed Graph: In-Degree (Red) & Out-Degree (Blue)",
    fontsize=14,
    fontweight="bold",
    pad=20,
)
plt.axis("off")
plt.tight_layout()
plt.show()

```

8-Vertex Directed Graph: In-Degree (Red) & Out-Degree (Blue)



In the example above, we can see above that vertex B has 3 total connections: 1 in-degree (denoted in red) and 2 out-degree (denoted in blue). Vertex B would be denoted as $\text{deg}^-(1)$ and $\text{deg}^+(2)$. Another example: vertex H has 3 total connections: 3 in-degrees (denoted in red) and 0 out-degrees (denoted in blue). Vertex H is denoted as $\text{deg}^-(3)$ and $\text{deg}^+(0)$.

5.3: Handshaking Theorem

Generally, degree values of all types of graphs can range from 0 to $|V| - 1$, where $|V|$ is the number of vertices within the graph. If a vertex has degree 0, then we say that the vertex is isolated from the rest of the graph. Additionally, if the vertex has degree 1, then we say that the vertex is a pendant. If we expand upon this and make a note of all the adjacent vertices, the formal definition is called a neighborhood. If we want to quickly calculate the total number of degrees of a simple graph, we can use what's called the Handshaking Theorem. The Handshaking Theorem states that since each edge contributes twice (one for initial and one for the terminal vertex), the sum of degrees is equal to twice the number of edges. The Handshaking Theorem also has an interesting result: the number of vertices with odd degree is even. Let us observe one last property with the Handshaking Theorem for directed graphs: then we can say that the sum of $\text{deg}^-(u)$ is equal to the sum of $\text{deg}^+(u)$, which is ultimately equal to the number of edges in the graph. Let us take a look at an example where we can see all the properties previously mentioned and the Handshaking Theorem.

```
In [ ]: import matplotlib.pyplot as plt
import networkx as nx

# 1. Create an undirected graph
G = nx.Graph()

# 2. Add 16 vertices labeled A through P
nodes = [chr(i) for i in range(ord("A"), ord("P") + 1)]
G.add_nodes_from(nodes)
```

```

# 3. Add edges to create isolated nodes (A, B), a pendant node (C), and connected c
edges = [
    # Pendant connection (Degree 1)
    ("C", "D"),
    # Core cluster
    ("D", "E"),
    ("D", "F"),
    ("E", "F"),
    ("E", "G"),
    ("F", "G"),
    # Middle network
    ("G", "H"),
    ("H", "I"),
    ("H", "J"),
    ("I", "J"),
    ("I", "K"),
    ("J", "L"),
    # Right cluster
    ("K", "M"),
    ("K", "N"),
    ("L", "N"),
    ("M", "N"),
    ("N", "O"),
    ("O", "P"),
]
G.add_edges_from(edges)

# 4. Calculate degrees for every node
degrees = dict(G.degree())
total_degrees = sum(degrees.values())
num_edges = G.number_of_edges()

# Count odd and even degree vertices
odd_degree_nodes = [node for node, deg in degrees.items() if deg % 2 != 0]
even_degree_nodes = [node for node, deg in degrees.items() if deg % 2 == 0]

# 5. Fixed positions across 16 nodes for a clean, readable layout
pos = {
    "A": (-3.0, 1.2), # Isolated
    "B": (-3.0, -1.2), # Isolated
    "C": (-2.0, 0.0), # Pendant (Degree 1)
    "D": (-1.2, 0.0),
    "E": (-0.6, 0.8),
    "F": (-0.6, -0.8),
    "G": (0.0, 0.0),
    "H": (0.6, 0.0),
    "I": (1.2, 0.8),
    "J": (1.2, -0.8),
    "K": (2.0, 0.8),
    "L": (2.0, -0.8),
    "M": (2.8, 1.2),
    "N": (2.8, 0.0),
    "O": (3.4, 0.0),
    "P": (4.0, 0.0),
}

```

```

plt.figure(figsize=(12, 6.5))

# 6. Draw nodes: Color odd-degree nodes light red and even-degree nodes light blue
nx.draw_networkx_nodes(
    G,
    pos,
    nodelist=even_degree_nodes,
    node_size=1200,
    node_color="#87CEEB",
    edgecolors="black",
    linewidths=1.2,
    label="Even Degree Node",
)
nx.draw_networkx_nodes(
    G,
    pos,
    nodelist=odd_degree_nodes,
    node_size=1200,
    node_color="#FFA07A",
    edgecolors="black",
    linewidths=1.2,
    label="Odd Degree Node",
)

# 7. Draw undirected edges
nx.draw_networkx_edges(G, pos, width=1.5, edge_color="#444444")

# 8. Draw Labels showing Node Name and Degree: Node\ndeg(v)
node_labels = {node: f"{node}\ndeg: {deg}" for node, deg in degrees.items()}
nx.draw_networkx_labels(
    G, pos, labels=node_labels, font_size=8, font_color="black", font_weight="bold"
)

plt.title(
    "16-Vertex Simple Undirected Graph: Handshaking Theorem Demonstration",
    fontsize=13,
    fontweight="bold",
    pad=15,
)

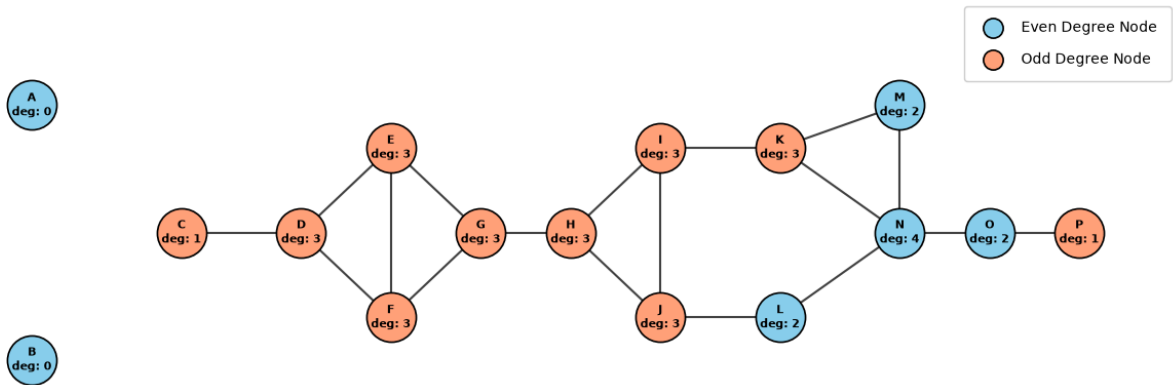
# 9. Clean, spaced-out Legend configuration
plt.legend(
    loc="upper right",
    framealpha=0.9,
    fontsize=10,
    markerscale=0.4, # Scales down circle symbols in the Legend
    labelspacing=1.2, # Adds vertical space between Legend rows
    handletextpad=1.0, # Adds horizontal space between circle and text
    borderpad=1.0, # Adds padding around the inner border of Legend
)

# Adjust plot boundaries to keep top and bottom clean
plt.ylim(-2.2, 2.2)
plt.axis("off")
plt.tight_layout()

```

```
plt.subplots_adjust(bottom=0.22, top=0.92)
plt.show()
```

16-Vertex Simple Undirected Graph: Handshaking Theorem Demonstration



In the example above, there are a total of 16 vertices, 18 edges, and 36 degrees. we can see that both vertices A and B are isolated since $\deg(0)$. We can also see that vertices C and P are pendants since $\deg(1)$. According to the Handshake Theorem which we previously defined:

$$\underbrace{\sum_{v \in V} \deg(v)}_{\text{Sum of all vertex degrees}} = \underbrace{2|E|}_{\text{Twice the number of edges}}$$

Substituting the values into the equation, we can see that the Handshake Theorem holds true:

$$\sum_{v \in V} \deg(v) = 2|E| \rightarrow 36 = 36$$

6: Paths and Connectivity

6.1: Paths Definition

What if instead of knowing how many edges are connected to a vertex, we want to know if a route exists from vertex v_0 to v_i ? First, there is a base definition that we must define where we do not care if the vertices have to be distinct, in which we call it a Walk. Further extending the idea, a closed walk is called a circuit ($v_0 = v_i$). When the length from v_0 to v_i is a consecutive sequence of distinct vertices, it is known as a path. Symbolically, it's represented as: $\{v_0, v_1, \dots, v_{i-1}, v_i\}$, where v_i and v_{i+1} are adjacent. Simply put, a path is a walk with distinct vertices. We refer to this type of path as a simple path since the same vertex does not appear twice. Further extending the idea, a closed path is called a cycle since there are no repeated vertices and a cycle graph is one where all the degrees are equal.

6.2: Connectivity Definition

Now that we have established the definition of a path, we can also define connectivity. If there exists a path from vertex u to vertex v , then we can confirm that v is reachable from u ($u \rightarrow v$), which we can conclude that u is connected to v . Expanded broadly to the whole graph, we can conclude that a whole graph is connected if there exists an x - y path for all vertices within the graph. We can denote the number of connected graph or components with the notation $K(G)$, where $K(G)$ represents the number of connected components. Let us look at the example below to see both cycles and connectivity.

```
In [ ]: import matplotlib.pyplot as plt
import networkx as nx

# 1. Create an undirected graph
G = nx.Graph()

# 2. Add 16 vertices labeled A through P
nodes = [chr(i) for i in range(ord("A"), ord("P") + 1)]
G.add_nodes_from(nodes)

# 3. Add edges to form 3 distinct connected components (Discontinuous Graph)
edges = [
    # Component 1 (Vertices A, B, C, D, E)
    ("A", "B"),
    ("B", "D"),
    ("D", "E"),
    ("E", "C"),
    ("C", "A"),
    # Component 2 (Vertices F, G, H, I, J)
    ("F", "G"),
    ("G", "H"),
    ("H", "I"),
    ("I", "J"),
    ("G", "I"),
    # Component 3 (Vertices K, L, M, N, O, P)
    ("K", "L"),
    ("L", "M"),
    ("M", "N"),
    ("N", "O"),
    ("O", "P"),
    ("M", "P"),
    ("K", "N"),
]
G.add_edges_from(edges)

# 4. Extract connected components
components = list(nx.connected_components(G))
num_components = len(components)

# Define distinct colors for the 3 components
component_colors = ["#87CEEB", "#FFA07A", "#98FB98"] # Sky Blue, Light Salmon, Pal

# 5. Fixed positions to visually separate the 3 components across the layout
pos = {
    # Component 1 (Left Cluster)
```

```

    "A": (-3.2, 0.8),
    "B": (-3.2, -0.8),
    "C": (-2.2, 0.8),
    "D": (-2.2, -0.8),
    "E": (-1.4, 0.0),
    # Component 2 (Center Cluster)
    "F": (-0.4, 1.0),
    "G": (-0.4, -0.2),
    "H": (0.4, 1.0),
    "I": (0.4, -0.2),
    "J": (0.0, -1.0),
    # Component 3 (Right Cluster)
    "K": (1.4, 0.8),
    "L": (1.4, -0.8),
    "M": (2.2, 0.8),
    "N": (2.2, -0.8),
    "O": (3.0, 0.8),
    "P": (3.0, -0.8),
}

plt.figure(figsize=(12, 6.5))

# 6. Draw each component with a unique color and Label for the Legend
for idx, comp in enumerate(components):
    comp_nodes = list(comp)
    nx.draw_networkx_nodes(
        G,
        pos,
        nodelist=comp_nodes,
        node_size=1200,
        node_color=component_colors[idx],
        edgecolors="black",
        linewidths=1.2,
        label=f"Component {idx + 1} ({len(comp_nodes)} Vertices)",
    )

# 7. Draw straight, clean edges
nx.draw_networkx_edges(G, pos, width=1.5, edge_color="#444444")

# 8. Draw node Labels (Letter names) inside the nodes
node_labels = {node: node for node in G.nodes()}
nx.draw_networkx_labels(
    G, pos, labels=node_labels, font_size=10, font_color="black", font_weight="bold"
)

plt.title(
    "16-Vertex Undirected Graph",
    fontsize=13,
    fontweight="bold",
    pad=15,
)

# 9. Clean, spaced-out Legend configuration
plt.legend(
    loc="upper right",
    framealpha=0.9,

```

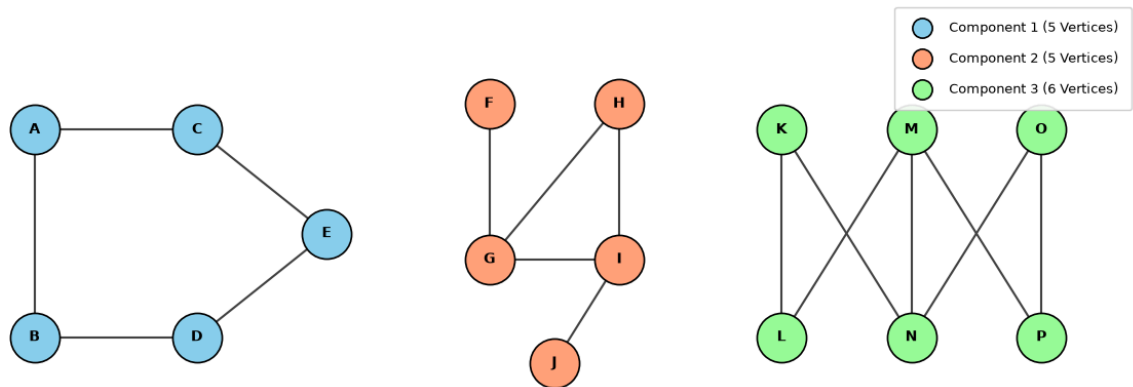
```

    fontsize=9.5,
    markerscale=0.4,
    labelspace=1.2,
    handletextpad=1.0,
    borderpad=1.0,
)

# Layout bounds
plt.xlim(-3.8, 3.6)
plt.ylim(-1.8, 1.8)
plt.axis("off")
plt.tight_layout()
plt.subplots_adjust(bottom=0.22, top=0.92)
plt.show()

```

16-Vertex Undirected Graph



In the above example, there are 3 total components within this 16-vertex graph, which can be denoted as $K(3)$. Since $K(G) > 1$, we can conclude that the graph overall is disconnected. To imagine a graph that is connected, each of the components on its own is connected, which we will analyze individually.

Component 1:

Analyzing component 1 individually, we can see that it is a connected undirected graph. There is a Walk from $A \rightarrow E$ and also a path since all of the vertices are unique. This is a special type of graph, called a "cycle graph," since for all vertices have the same number of degrees: $\deg(2)$.

Component 2:

Analyzing component 2 individually, we can see that it is a connected undirected graph. One possible Walk from $F \rightarrow J$ is $F \rightarrow G \rightarrow H \rightarrow I \rightarrow J$. This Walk can also be considered a Path since all of the vertices are unique and are not repeated. One possible path from $G \rightarrow G$ is $G \rightarrow H \rightarrow I \rightarrow G$, which we can define as a closed path, also known as a cycle.

Component 3:

Analyzing component 3 individually, we can see that it is a connected undirected graph. One possible Walk from $K \rightarrow L$ is $K \rightarrow N \rightarrow M \rightarrow P \rightarrow O \rightarrow N \rightarrow M \rightarrow L$. This would NOT be a path since the vertices are not unique and are repeated. A possible path from $K \rightarrow L$ is $K \rightarrow N \rightarrow O \rightarrow P \rightarrow M \rightarrow L$.

6.3: Properties of Connectivity

Some important graph relation terms we will define are symmetric, reflexive, and transitive.

Symmetric Definition:

A graph is symmetric (arc-transitive) if, for any two pairs of adjacent vertices (u, v) and (x, y) are mapped to each other. Specifically, there has to be graph automorphism $f: V(G) \rightarrow V(G)$ such that $f(u) = x$ and $f(v) = y$ simultaneously. The formal mathematical definition is listed below:

A relation on A is symmetric if $\forall x, y \in A$, if xRy then yRx
where xRy means X related to Y and yRx means Y related to X

Symmetric undirected and directed graphs can be seen below:

```
In [ ]: import matplotlib.pyplot as plt
import networkx as nx

# Create figure with 2 subplots side-by-side
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

# =====
# 1. UNDIRECTED GRAPH (2 Symmetric Vertices)
# =====
G_undirected = nx.Graph()
G_undirected.add_edge("A", "B")

pos_undirected = {"A": (-1.0, 0.0), "B": (1.0, 0.0)}

# Draw Undirected Nodes
nx.draw_networkx_nodes(
    G_undirected,
    pos_undirected,
    ax=ax1,
    node_size=2000,
    node_color="#87CEEB",
    edgecolors="black",
    linewidths=1.5,
)

# Draw Undirected Edge
nx.draw_networkx_edges(
    G_undirected, pos_undirected, ax=ax1, width=2.5, edge_color="#333333"
)

# Draw Labels
```

```

nx.draw_networkx_labels(
    G_undirected,
    pos_undirected,
    ax=ax1,
    font_size=14,
    font_color="black",
    font_weight="bold",
)

ax1.set_title(
    "1. Undirected Graph (2 Vertices)\nSymmetric: f(A)=B, f(B)=A",
    fontsize=12,
    fontweight="bold",
    pad=15,
)

ax1.set_xlim(-2.0, 2.0)
ax1.set_ylim(-1.0, 1.0)
ax1.axis("off")

# =====
# 2. DIRECTED GRAPH (2 Symmetric Vertices)
# =====
# To be symmetric, A and B must point to each other symmetrically (A <-> B)
G_directed = nx.DiGraph()
G_directed.add_edges_from([("A", "B"), ("B", "A")])

pos_directed = {"A": (-1.0, 0.0), "B": (1.0, 0.0)}

# Draw Directed Nodes
nx.draw_networkx_nodes(
    G_directed,
    pos_directed,
    ax=ax2,
    node_size=2000,
    node_color="#FFA07A",
    edgecolors="black",
    linewidths=1.5,
)

# Draw Curved Directed Arrows so both directions (A->B and B->A) are clear
nx.draw_networkx_edges(
    G_directed,
    pos_directed,
    ax=ax2,
    width=2.0,
    edge_color="#333333",
    arrowstyle="->|>",
    arrowsize=20,
    node_size=2000,
    connectionstyle="arc3,rad=0.2", # Curved arcs for bidirectional edges
)

# Draw Labels
nx.draw_networkx_labels(
    G_directed,

```

```

pos_directed,
ax=ax2,
font_size=14,
font_color="black",
font_weight="bold",
)

ax2.set_title(
    "2. Directed Graph (2 Vertices)\nSymmetric: f(A)=B, f(B)=A (Mutual Edges)",
    fontsize=12,
    fontweight="bold",
    pad=15,
)

ax2.set_xlim(-2.0, 2.0)
ax2.set_ylim(-1.0, 1.0)
ax2.axis("off")

plt.tight_layout()
plt.subplots_adjust(bottom=0.25)
plt.show()

```

1. Undirected Graph (2 Vertices)
Symmetric: f(A)=B, f(B)=A



2. Directed Graph (2 Vertices)
Symmetric: f(A)=B, f(B)=A (Mutual Edges)



Reflexive Definition

A graph is reflexive if for all the vertices u in the graph, there exists an edge (u, u) , which is essentially a self-loop that connects to itself. The formal mathematical definition is listed below:

A relation A is reflexive if: $xRx, \forall x \in A$
 $\setminus \text{small}\{\text{where } xRx \text{ means } X \text{ related to } X\}$

Reflexive undirected and directed graphs can be seen below:

```

In [ ]: import matplotlib.pyplot as plt
import networkx as nx

# Create figure with 2 subplots side-by-side
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4.5))

# =====
# 1. UNDIRECTED REFLEXIVE GRAPH (Single Node)
# =====
G_undirected = nx.Graph()

```

```

G_undirected.add_node("A")
G_undirected.add_edge("A", "A") # Self-Loop

pos_u = {"A": (0, 0)}

# Draw Undirected Node
nx.draw_networkx_nodes(
    G_undirected,
    pos_u,
    ax=ax1,
    node_size=2500,
    node_color="#87CEEB",
    edgecolors="black",
    linewidths=1.5,
)

# Draw Self-Loop Edge
nx.draw_networkx_edges(
    G_undirected, pos_u, ax=ax1, width=2.0, edge_color="#333333"
)

# Draw Node Label
nx.draw_networkx_labels(
    G_undirected,
    pos_u,
    ax=ax1,
    font_size=14,
    font_color="black",
    font_weight="bold",
)

ax1.set_title(
    "1. Undirected Reflexive Graph\nSingle Vertex with Self-Loop {A, A}",
    fontsize=11,
    fontweight="bold",
    pad=15,
)

ax1.set_xlim(-1.5, 1.5)
ax1.set_ylim(-1.5, 1.5)
ax1.axis("off")

# =====
# 2. DIRECTED REFLEXIVE GRAPH (Single Node)
# =====
G_directed = nx.DiGraph()
G_directed.add_node("A")
G_directed.add_edge("A", "A") # Directed self-loop (A -> A)

pos_d = {"A": (0, 0)}

# Draw Directed Node
nx.draw_networkx_nodes(
    G_directed,
    pos_d,
    ax=ax2,

```

```

    node_size=2500,
    node_color="#FFA07A",
    edgecolors="black",
    linewidths=1.5,
)

# Draw Directed Self-Loop Arrow
nx.draw_networkx_edges(
    G_directed,
    pos_d,
    ax=ax2,
    width=2.0,
    edge_color="#333333",
    arrowstyle="->",
    arrowsize=20,
    node_size=2500,
)

# Draw Node Label
nx.draw_networkx_labels(
    G_directed,
    pos_d,
    ax=ax2,
    font_size=14,
    font_color="black",
    font_weight="bold",
)

ax2.set_title(
    "2. Directed Reflexive Graph\nSingle Vertex with Self-Loop ( $A \rightarrow A$ )",
    fontsize=11,
    fontweight="bold",
    pad=15,
)

ax2.set_xlim(-1.5, 1.5)
ax2.set_ylim(-1.5, 1.5)
ax2.axis("off")

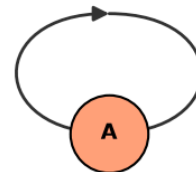
plt.tight_layout()
plt.subplots_adjust(bottom=0.28)
plt.show()

```

1. Undirected Reflexive Graph
Single Vertex with Self-Loop $\{A, A\}$



2. Directed Reflexive Graph
Single Vertex with Self-Loop ($A \rightarrow A$)



Transitive Definition:

A graph is said to be transitive if there exists a path from vertices X to Y and from Y to Z, then there also must be an edge from X to Z. The formal mathematical definition is listed below:

A relation on A is transitive if $\forall x, y, z \in A$, if xRy and yRz $\implies xRz$ where xRy means X related to Y, yRz means Y related to Z, and xRz means X related to Z.

Transitive undirected and directed graphs can be seen below:

```
In [ ]: import matplotlib.pyplot as plt
import networkx as nx

# Create figure with 2 subplots side-by-side
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

# Triangular Layout positions for 3 vertices (X, Y, Z)
pos = {
    "X": (-1.0, -0.8), # Bottom-Left
    "Y": (0.0, 0.8), # Top-Center
    "Z": (1.0, -0.8), # Bottom-Right
}

# =====
# 1. UNDIRECTED TRANSITIVE GRAPH (K3 Triangle)
# =====
G_undirected = nx.Graph()
G_undirected.add_nodes_from(["X", "Y", "Z"])

# In undirected graphs, transitivity requires that if {X, Y} and {Y, Z} exist, {X, Z} must also exist.
# This forms a complete 3-clique (triangle graph K3).
G_undirected.add_edges_from([("X", "Y"), ("Y", "Z"), ("X", "Z")])

# Draw Undirected Nodes
nx.draw_networkx_nodes(
    G_undirected,
    pos,
    ax=ax1,
    node_size=2000,
    node_color="#87CEEB",
    edgecolors="black",
    linewidths=1.5,
)

# Draw Undirected Edges
nx.draw_networkx_edges(
    G_undirected, pos, ax=ax1, width=2.0, edge_color="#333333"
)

# Draw Node Labels
nx.draw_networkx_labels(
    G_undirected,
    pos,
    ax=ax1,
```

```

        font_size=14,
        font_color="black",
        font_weight="bold",
    )

    ax1.set_title(
        "1. Undirected Transitive Graph\n{X, Y} and {Y, Z}  $\Rightarrow$  {X, Z} Exists ( $K_3$ )",
        fontsize=11,
        fontweight="bold",
        pad=15,
    )
    ax1.set_xlim(-1.6, 1.6)
    ax1.set_ylim(-1.3, 1.3)
    ax1.axis("off")

# =====
# 2. DIRECTED TRANSITIVE GRAPH (TOSET Chain)
# =====
G_directed = nx.DiGraph()
G_directed.add_nodes_from(["X", "Y", "Z"])

# Transitive property: X -> Y and Y -> Z implies shortcut X -> Z
G_directed.add_edges_from(
    [
        ("X", "Y"), # X -> Y
        ("Y", "Z"), # Y -> Z
        ("X", "Z"), # X -> Z (Transitive shortcut edge)
    ]
)

# Draw Directed Nodes
nx.draw_networkx_nodes(
    G_directed,
    pos,
    ax=ax2,
    node_size=2000,
    node_color="#FFA07A",
    edgecolors="black",
    linewidths=1.5,
)

# Draw Directed Arrows (Straight Lines for clear hierarchy)
nx.draw_networkx_edges(
    G_directed,
    pos,
    ax=ax2,
    width=2.0,
    edge_color="#333333",
    arrowstyle="-|>",
    arrowsize=20,
    node_size=2000,
    connectionstyle="arc3,rad=0.0", # Straight Lines
)

# Draw Node Labels

```

```

nx.draw_networkx_labels(
    G_directed,
    pos,
    ax=ax2,
    font_size=14,
    font_color="black",
    font_weight="bold",
)

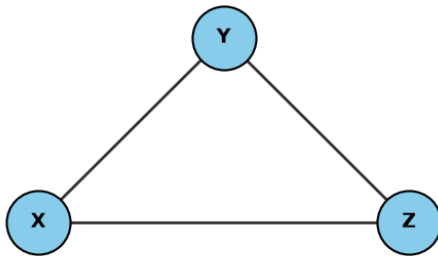
ax2.set_title(
    "2. Directed Transitive Graph\n( $X \rightarrow Y$ ) and ( $Y \rightarrow Z$ )  $\Rightarrow$  ( $X \rightarrow Z$ ) Shortcut Edge",
    fontsize=11,
    fontweight="bold",
    pad=15,
)

ax2.set_xlim(-1.6, 1.6)
ax2.set_ylim(-1.3, 1.3)
ax2.axis("off")

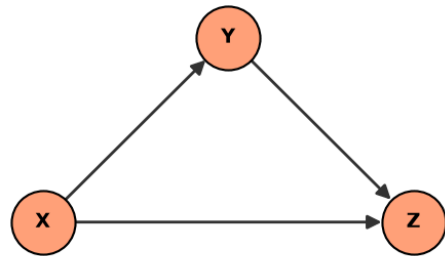
plt.tight_layout()
plt.subplots_adjust(bottom=0.28)
plt.show()

```

1. Undirected Transitive Graph
 $\{X, Y\}$ and $\{Y, Z\} \Rightarrow \{X, Z\}$ Exists (K_3)



2. Directed Transitive Graph
 $(X \rightarrow Y)$ and $(Y \rightarrow Z) \Rightarrow (X \rightarrow Z)$ Shortcut Edge



6.4: Connectivity in Directed Graphs

What about connectivity in the case of directed graphs? Since edges have direction, connectivity depends on whether paths exist in one or both directions between vertices.

Strongly Connected Graphs:

Vertices u and v are strongly connected if there exists a directed path $u \rightarrow v$ AND a directed path $v \rightarrow u$. Strong connectivity defines an equivalence relation on the vertex set, partitioning the graph into maximal strongly connected components. Therefore, a graph G is strongly connected if every vertex is reachable from every other vertex.

Unilaterally Connected Graphs:

A graph G is unilaterally connected if, for every pair of vertices u and v , there exists a directed path from $u \rightarrow v$ OR from $v \rightarrow u$. Simply put, for any two vertices you pick, you can always walk from one to the other in at least one direction—it does not require a bidirectional path between every pair. When represented as a path (reachability) matrix

ordered by topological flow, the matrix will yield a structure where all entries in either the upper triangle OR the lower triangle are 1s.

Weakly Connected Graphs:

A directed graph G is weakly connected if replacing all of its directed edges with undirected edges results in a connected undirected graph (i.e., it forms a single connected component). Weak connectivity is the most lenient form of directed connectivity: if a graph is strongly or unilaterally connected, it is inherently weakly connected, but the reverse is not necessarily true.

Summary & Intuitive Analogies

- **Strongly Connected:** $u \rightarrow v$ **AND** $v \rightarrow u$. Think of a city with two-way streets connecting every single location.
- **Unilaterally Connected:** $u \rightarrow v$ **OR** $v \rightarrow u$. Think of a one-way street system where you can always travel from any building u to building v (or vice-versa), but you might not be able to drive back along the same route.
- **Weakly Connected:** Connected only when arrow directions are ignored. Think of a road network where traffic flow rules are temporarily suspended so all roads become two-way.

```
In [ ]: import matplotlib.pyplot as plt
import networkx as nx

# Create a figure with 3 subplots side-by-side
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(15, 5))

# Standard triangle layout for 3 vertices (A, B, C)
pos = {"A": (-0.8, -0.6), "B": (0.0, 0.8), "C": (0.8, -0.6)}

# =====
# 1. STRONGLY CONNECTED GRAPH
# =====
# Path exists in both directions for all pairs (3-cycle loop: A -> B -> C -> A)
G_strong = nx.DiGraph()
G_strong.add_nodes_from(["A", "B", "C"])
G_strong.add_edges_from([("A", "B"), ("B", "C"), ("C", "A")])

nx.draw_networkx_nodes(
    G_strong,
    pos,
    ax=ax1,
    node_size=1600,
    node_color="#87CEEB",
    edgecolors="black",
    linewidths=1.2,
)
nx.draw_networkx_edges(
```

```

    G_strong,
    pos,
    ax=ax1,
    width=2.0,
    edge_color="#333333",
    arrowstyle="-|>",
    arrowsize=18,
    node_size=1600,
    connectionstyle="arc3,rad=0.1",
)
nx.draw_networkx_labels(
    G_strong, pos, ax=ax1, font_size=12, font_weight="bold"
)

ax1.set_title(
    "1. Strongly Connected\n(A ↔ B, B ↔ C, A ↔ C)",
    fontsize=11,
    fontweight="bold",
    pad=12,
)
ax1.set_xlim(-1.3, 1.3)
ax1.set_ylim(-1.1, 1.2)
ax1.axis("off")

# =====
# 2. UNILATERALLY CONNECTED GRAPH
# =====
# One-way path sequence (A → B → C).
# Every pair has a path in AT LEAST ONE direction (e.g., A → C via B), but not bot
G_unilateral = nx.DiGraph()
G_unilateral.add_nodes_from(["A", "B", "C"])
G_unilateral.add_edges_from([("A", "B"), ("B", "C")])

nx.draw_networkx_nodes(
    G_unilateral,
    pos,
    ax=ax2,
    node_size=1600,
    node_color="#FFA07A",
    edgecolors="black",
    linewidths=1.2,
)
nx.draw_networkx_edges(
    G_unilateral,
    pos,
    ax=ax2,
    width=2.0,
    edge_color="#333333",
    arrowstyle="-|>",
    arrowsize=18,
    node_size=1600,
)
nx.draw_networkx_labels(
    G_unilateral, pos, ax=ax2, font_size=12, font_weight="bold"
)

```

```

ax2.set_title(
    "2. Unilaterally Connected\n(A → B → C, but C →/A)",
    fontsize=11,
    fontweight="bold",
    pad=12,
)
ax2.set_xlim(-1.3, 1.3)
ax2.set_ylim(-1.1, 1.2)
ax2.axis("off")

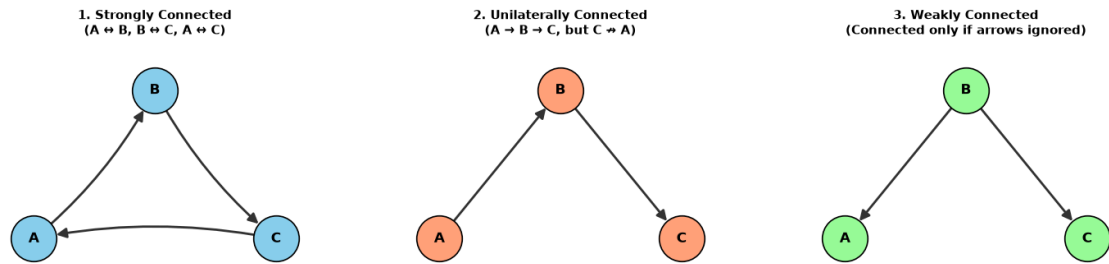
# =====
# 3. WEAKLY CONNECTED GRAPH
# =====
# Outer arrows point away from B (B → A and B → C).
# No directed path exists between A and C in EITHER direction (A →/C and C →/A).
# Connected ONLY if arrow directions are ignored.
G_weak = nx.DiGraph()
G_weak.add_nodes_from(["A", "B", "C"])
G_weak.add_edges_from([("B", "A"), ("B", "C")])

nx.draw_networkx_nodes(
    G_weak,
    pos,
    ax=ax3,
    node_size=1600,
    node_color="#98FB98",
    edgecolors="black",
    linewidths=1.2,
)
nx.draw_networkx_edges(
    G_weak,
    pos,
    ax=ax3,
    width=2.0,
    edge_color="#333333",
    arrowstyle="-|>",
    arrowsize=18,
    node_size=1600,
)
nx.draw_networkx_labels(G_weak, pos, ax=ax3, font_size=12, font_weight="bold")

ax3.set_title(
    "3. Weakly Connected\n(Connected only if arrows ignored)",
    fontsize=11,
    fontweight="bold",
    pad=12,
)
ax3.set_xlim(-1.3, 1.3)
ax3.set_ylim(-1.1, 1.2)
ax3.axis("off")

plt.tight_layout()
plt.subplots_adjust(bottom=0.28)
plt.show()

```



6.5: Additional Terms for Directed Graph Connectivity

Power of a Directed Graph: The k th-power of a directed graph can be defined as a "shortcut" line from vertex u to vertex v iff you can get from u to v in exactly k steps. For example:

- G^0 : A vertex u has a self-loop to itself.
- G^1 : Direct connections using 1 step.
- G^2 : Direct connections between two vertices that take exactly 2 steps.
- G^3 : Direct connections between two vertices that take exactly 3 steps.
- G^k : Direct connections between two vertices that take exactly k steps.

Transitive Closure: Combines all of the powers ($G^0 \cup G^1 \cup G^2 \cup G^3 \cup G^{kth}$) into a single graph. The notation for transitive closure is $G^* = \bigcup_{k=0}^{\infty} G^k$ and can be more informally thought of as "the ultimate reachability map."

```
In [ ]: import matplotlib.pyplot as plt
import networkx as nx

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Fixed horizontal layout for 4 vertices
pos = {"A": (-1.5, 0), "B": (-0.5, 0), "C": (0.5, 0), "D": (1.5, 0)}

# =====
# 1. SECOND POWER OF GRAPH G ( $G^2$ )
# =====
# Path of Length 2: A->C (via B) and B->D (via C)
G_squared = nx.DiGraph()
G_squared.add_nodes_from(["A", "B", "C", "D"])
G_squared.add_edges_from([("A", "C"), ("B", "D")])

nx.draw_networkx_nodes(
    G_squared,
    pos,
    ax=ax1,
    node_size=1800,
    node_color="#87CEEB",
    edgecolors="black",
    linewidths=1.2,
)
nx.draw_networkx_edges(
    G_squared,
```

```

    pos,
    ax=ax1,
    width=2.0,
    edge_color="#333333",
    arrowstyle="-|>",
    arrowsize=18,
    node_size=1800,
    connectionstyle="arc3,rad=-0.3", # Curved arcs for readability
)
nx.draw_networkx_labels(
    G_squared, pos, ax=ax1, font_size=12, font_weight="bold"
)

ax1.set_title(
    "1. Second Power  $G^2$  (Length = 2)\nEdges exist ONLY for 2-step hops (A→C, B→D)",
    fontsize=11,
    fontweight="bold",
    pad=15,
)
ax1.set_xlim(-2.0, 2.0)
ax1.set_ylim(-1.0, 1.0)
ax1.axis("off")

# =====
# 2. TRANSITIVE CLOSURE OF GRAPH G ( $G^*$ )
# =====
#  $G^*$  contains self-loops ( $k=0$ ), 1-step (A→B, B→C, C→D), 2-step (A→C, B→D), and
G_star = nx.DiGraph()
G_star.add_nodes_from(["A", "B", "C", "D"])

# Edges for  $k=0$  (Self-Loops)
self_loops = [("A", "A"), ("B", "B"), ("C", "C"), ("D", "D")]
# Edges for  $k \geq 1$  (ALL reachable pairs)
reachable_edges = [
    ("A", "B"),
    ("B", "C"),
    ("C", "D"), # k=1
    ("A", "C"),
    ("B", "D"), # k=2
    ("A", "D"), # k=3
]

G_star.add_edges_from(self_loops + reachable_edges)

nx.draw_networkx_nodes(
    G_star,
    pos,
    ax=ax2,
    node_size=1800,
    node_color="#FFA07A",
    edgecolors="black",
    linewidths=1.2,
)

# Draw non-self-loop edges with varying arc curvatures

```

```

edge_rads = {
    ("A", "B"): 0.0,
    ("B", "C"): 0.0,
    ("C", "D"): 0.0, # Straight lines for k=1
    ("A", "C"): -0.3,
    ("B", "D"): -0.3, # Curved arcs for k=2
    ("A", "D"): -0.5, # Higher arc for k=3
}

for u, v in reachable_edges:
    nx.draw_networkx_edges(
        G_star,
        pos,
        ax=ax2,
        edgelist=[(u, v)],
        width=1.8,
        edge_color="#333333",
        arrowstyle="-|>",
        arrowsize=16,
        node_size=1800,
        connectionstyle=f"arc3,rad={edge_rads[(u,v)]}",
    )

# Draw Self-Loops (k=0)
nx.draw_networkx_edges(
    G_star,
    pos,
    ax=ax2,
    edgelist=self_loops,
    width=1.5,
    edge_color="#d95f02",
    arrowstyle="-|>",
    arrowsize=14,
    node_size=1800,
)

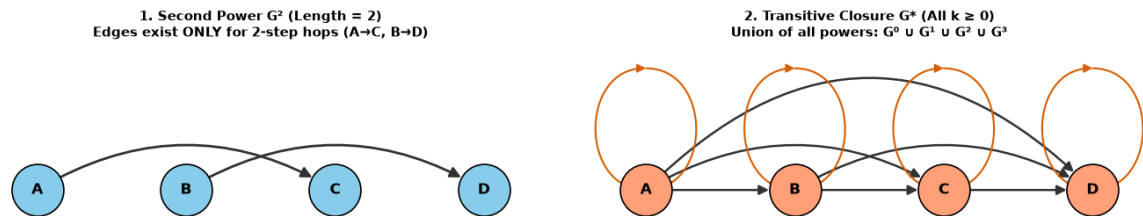
nx.draw_networkx_labels(G_star, pos, ax=ax2, font_size=12, font_weight="bold")

ax2.set_title(
    "2. Transitive Closure  $G^*$  (All  $k \geq 0$ )\nUnion of all powers:  $G^0 \cup G^1 \cup G^2 \cup G^3$ "
    fontsize=11,
    fontweight="bold",
    pad=15,
)

ax2.set_xlim(-2.0, 2.0)
ax2.set_ylim(-1.0, 1.0)
ax2.axis("off")

plt.tight_layout()
plt.subplots_adjust(bottom=0.25)
plt.show()

```



7: Important Types of Graphs

We will briefly cover the categories for classifying graphs: size, structure, direction, weights, special cases, density, connectivity, and cycles.

7.1: Based on Size

Finite Graphs:

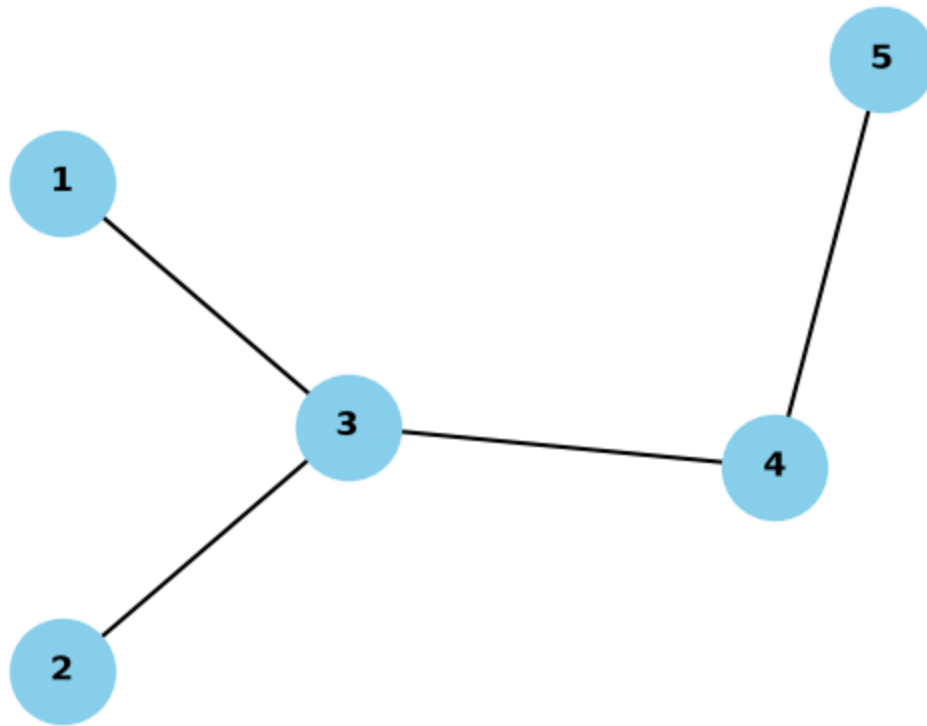
A finite graph is a graph with finite number of edges and vertices.

```
In [ ]: import matplotlib.pyplot as plt
import networkx as nx

# Define edges matching the layout
edges = [("1", "3"), ("2", "3"), ("3", "4"), ("4", "5")]
G = nx.Graph(edges)

# Exact vertex coordinates to match the image structure
pos = {
    "1": (-1.0, 0.6),
    "2": (-1.0, -0.6),
    "3": (-0.2, 0.0),
    "4": (1.0, -0.1),
    "5": (1.3, 0.9),
}

nx.draw_networkx(
    G,
    pos,
    node_color="#87CEEB",
    node_size=1400,
    font_weight="bold",
    width=1.5,
)
plt.axis("off")
plt.show()
```



Infinite Graphs An infinite graph is a graph with an infinite number of edges and vertices.

```

In [ ]: import matplotlib.pyplot as plt
import networkx as nx

# Create a smaller 2x3 grid graph (2 rows, 3 columns)
rows, cols = 2, 3
G = nx.grid_2d_graph(rows, cols)

# Define explicit 2D grid coordinates for positions
pos = {(r, c): (c, rows - 1 - r) for r in range(rows) for c in range(cols)}

fig, ax = plt.subplots(figsize=(7, 3.5))

# Draw inner grid edges with a custom dark slate color
edge_color = "#2C3E50"
nx.draw_networkx_edges(
    G, pos, ax=ax, width=1.8, edge_color=edge_color, style="solid"
)

# Draw extended "stub" lines along the boundary with smaller stubs
stub_length = 0.3
for r in range(rows):
    for c in range(cols):
        x, y = pos[(r, c)]
        # Top boundary stubs
        if r == 0:
            ax.plot([x, x], [y, y + stub_length], color=edge_color, lw=1.8)
        # Bottom boundary stubs
        if r == rows - 1:
            ax.plot([x, x], [y, y - stub_length], color=edge_color, lw=1.8)
        # Left boundary stubs

```

```

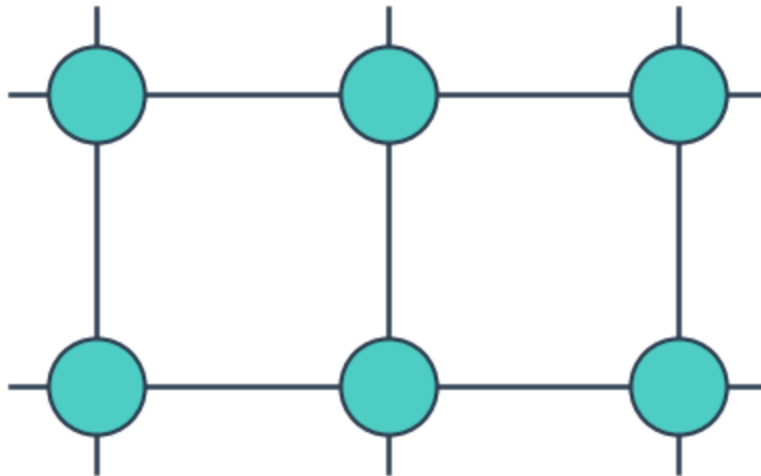
    if c == 0:
        ax.plot([x, x - stub_length], [y, y], color=edge_color, lw=1.8)
    # Right boundary stubs
    if c == cols - 1:
        ax.plot([x, x + stub_length], [y, y], color=edge_color, lw=1.8)

# Draw circular nodes with a soft teal color and dark border outline
nx.draw_networkx_nodes(
    G,
    pos,
    ax=ax,
    node_size=1200,
    node_color="#4ECDC4",
    edgecolors=edge_color,
    linewidths=1.5,
)

# Set equal aspect ratio and margins for the smaller canvas
ax.set_aspect("equal")
ax.set_xlim(-0.6, cols - 1 + 0.6)
ax.set_ylim(-0.6, rows - 1 + 0.6)
ax.axis("off")

plt.tight_layout()
plt.show()

```



7.2: Based on Structure

Trivial Graph:

A trivial graph contains only one vertex and no edges (also known as a singleton graph or a single vertex graph).

```
In [ ]: import matplotlib.pyplot as plt
import networkx as nx

# Create a singleton graph (1 node, 0 edges)
G = nx.Graph()
G.add_node("V")

nx.draw(G, with_labels=True, node_color="#87CEEB", node_size=1500)
plt.show()
```

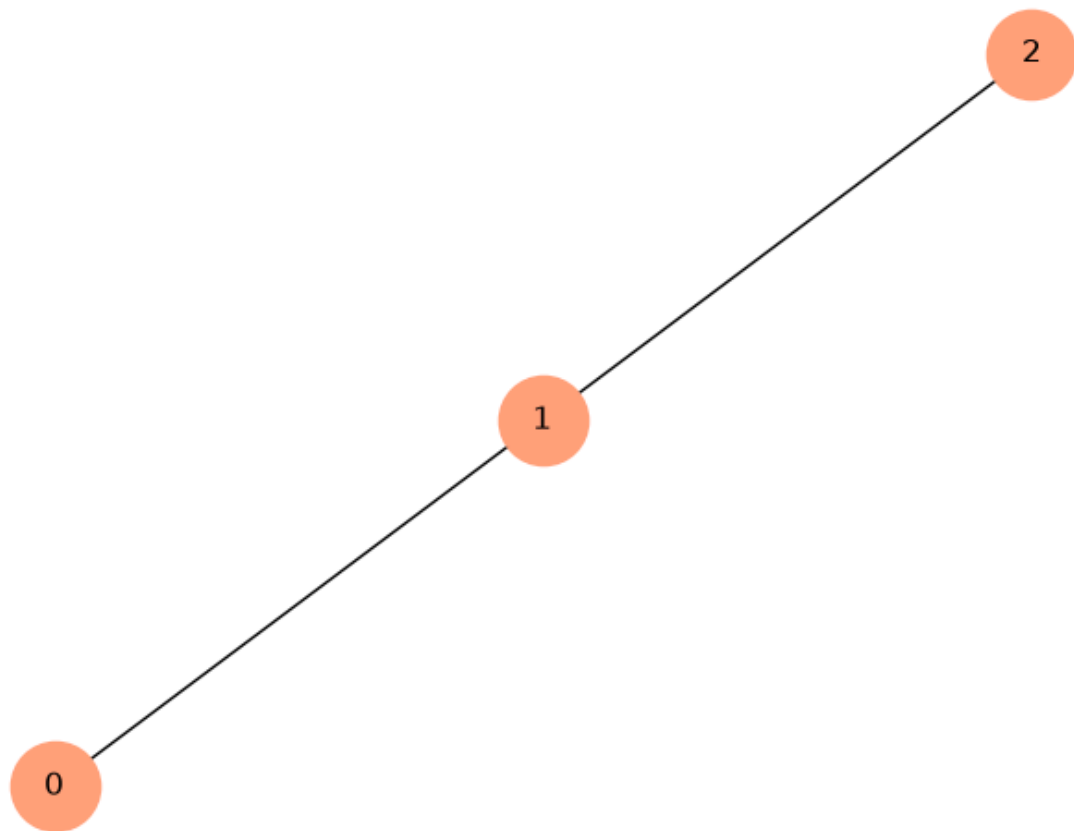


Simple Graph:

A simple graph does not have more than one edge between a pair of vertices anywhere in the graph.

```
In [ ]: import matplotlib.pyplot as plt
import networkx as nx

G = nx.path_graph(3)
nx.draw(G, with_labels=True, node_color="#FFA07A", node_size=1200)
plt.show()
```



Multi Graph:

A multigraph contains parallel edges but does not contain any self-loops.

- Parallel Edges: Two vertices that are connected with more than 1 edge

```
In [ ]: import matplotlib.pyplot as plt
import networkx as nx

# Create graph structure
G = nx.MultiGraph(
    [
        ("V1", "V2"),
        ("V1", "V3"),
        ("V2", "V4"),
        ("V1", "V4"),
        ("V1", "V4"),
        ("V1", "V5"),
    ]
)
pos = {
    "V5": (-1.5, 1.3),
    "V1": (-0.4, 0.5),
    "V2": (1.3, 0.5),
    "V4": (-0.4, -1.2),
    "V3": (1.3, -1.2),
}
```

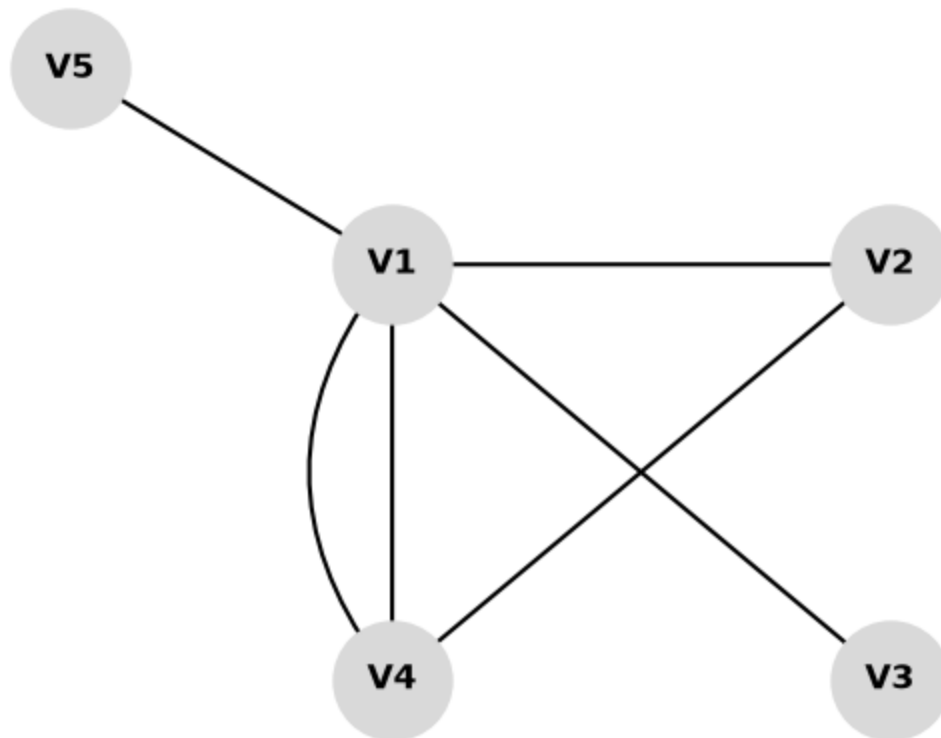
```

# Draw nodes and labels
nx.draw_networkx_nodes(G, pos, node_size=1800, node_color="#D9D9D9")
nx.draw_networkx_labels(G, pos, font_weight="bold")

# Draw straight edges and curved parallel edge
straight_edges = [
    ("V1", "V2"),
    ("V1", "V3"),
    ("V2", "V4"),
    ("V1", "V4"),
    ("V1", "V5"),
]
nx.draw_networkx_edges(G, pos, edgelist=straight_edges, width=1.5)
nx.draw_networkx_edges(
    G, pos, edgelist=[("V1", "V4")], width=1.5, connectionstyle="arc3,rad=0.4"
)

plt.axis("off")
plt.show()

```



Null Graph:

A null graph is a graph with only isolated vertices (no edges connecting the vertices). A null graph can also be known as an edgeless graph, an isolated graph, or a discrete graph. Special case: An order-0 null graph is a graph with 0 vertices and 0 edges.

```

In [ ]: import matplotlib.pyplot as plt
import networkx as nx

# Null graph: 4 vertices in a tight grid, 0 edges

```

```

G = nx.empty_graph(4)
pos = {0: (0, 1), 1: (1, 1), 2: (0, 0), 3: (1, 0)}

nx.draw(G, pos, with_labels=True, node_color="#D9D9D9", node_size=1200)
plt.xlim(-0.5, 1.5)
plt.ylim(-0.5, 1.5)
plt.show()

```



Complete Graph:

A Complete graph is a simple graph with the degree of all vertices are $n - 1$. A complete graph is also known as a Full Graph.

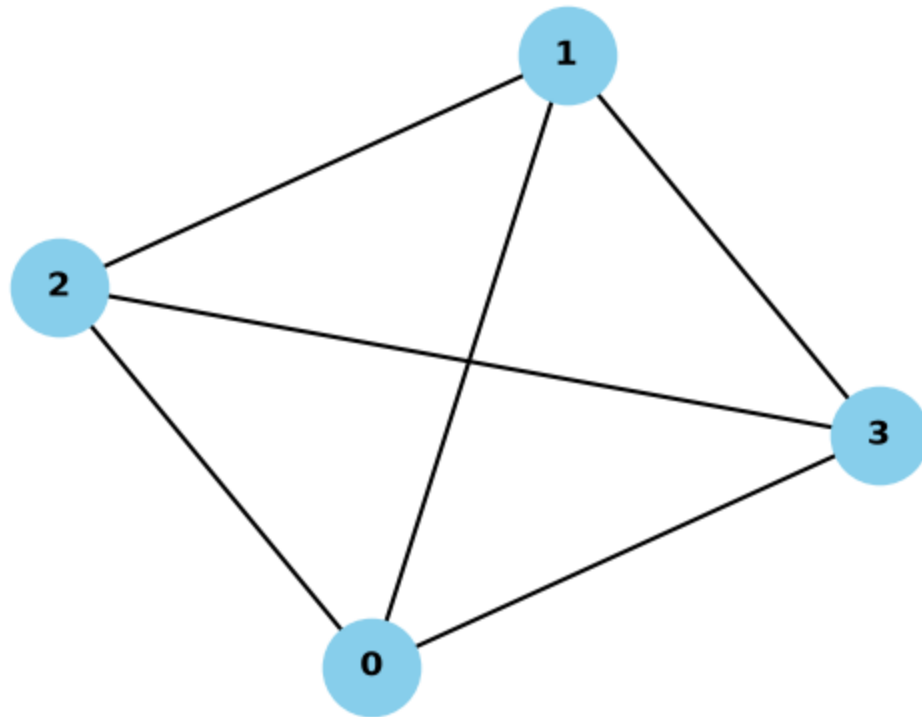
```

In [ ]: import matplotlib.pyplot as plt
import networkx as nx

# Create a small finite graph with 4 vertices and 5 edges
G = nx.complete_graph(4) # K_4 graph

nx.draw_networkx(
    G, node_color="#87CEEB", node_size=1200, font_weight="bold", width=1.5
)
plt.axis("off")
plt.show()

```



7.3: Based on Direction:

There are two types of direction graphs: directed and undirected. We have already gone over examples for directed and undirected graphs so they will just be briefly explained here.

Directed Graphs:

As previously mentioned, a Directed Graph is a graph where the edges have direction, where the arrows indicate the direction of travel.

Undirected Graphs:

As previously mentioned, an undirected graph is a graph where the edges do not have a specific direction, meaning that the edge connections can go both ways.

7.4: Based on Edge Weight

There are two types of edge-weighted graphs: Weighted and Unweighted. Edge-weighted graphs were already explained previously so they will just be briefly mentioned here.

Weighted Graphs:

As previously mentioned, a Weighted Graph is a graph where each edge has a unit of measurement that represents distance, cost, or time.

Unweighted Graphs:

As previously mentioned, an Unweighted Graph is a graph where each edge is treated equally, just showing connections between points.

7.5: Based on Connectivity

A graph can either be connected or disconnected. We have already gone over connected and disconnected graphs so they will just be briefly mentioned here.

Connected Graphs:

A Connected Graph is a graph where there exists at least one path between every pair of vertices in a graph.

Disconnected Graphs:

A Disconnected Graph is a graph where there are vertices that cannot be reached in a graph (i.e. a null graph).

7.6: Special Types of Graphs

We will note two special types of graphs: pseudographs and Regular graphs.

Pseudograph:

A pseudograph is a graph that allows self-loops and multiple edges.

```
In [ ]: import matplotlib.pyplot as plt
import networkx as nx

# Define graph with parallel edges and a self-loop
G = nx.MultiGraph(
    [
        ("1", "4"),
        ("1", "2"),
        ("1", "2"), # Parallel edge
        ("2", "3"),
        ("3", "4"),
        ("4", "4"), # Self-loop
    ]
)

pos = {"1": (0, 1), "4": (2, 1), "2": (0, 0), "3": (2, 0)}

# Draw base nodes, labels, straight edges, and self-loops
nx.draw_networkx_nodes(G, pos, node_color="#E9304E", node_size=1800)
nx.draw_networkx_labels(G, pos, font_weight="bold")
nx.draw_networkx_edges(
    G, pos, edgelist=[("1", "4"), ("1", "2"), ("2", "3"), ("3", "4")]
)

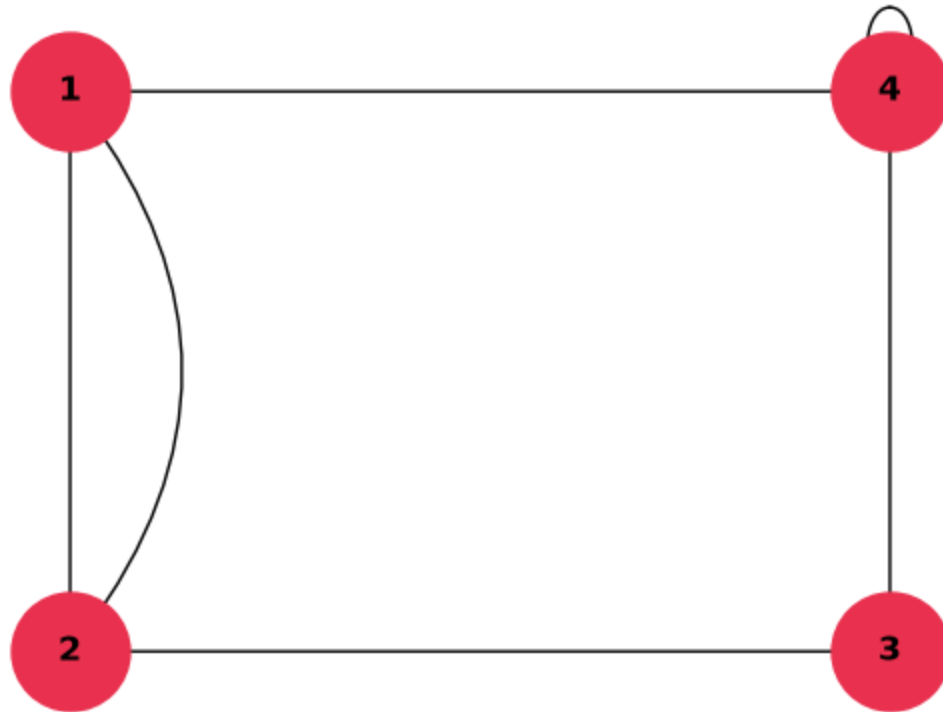
# Draw curved parallel edge (1 to 2) and self-loop (4)
nx.draw_networkx_edges(
    G, pos, edgelist=[("1", "2")], connectionstyle="arc3,rad=-0.4"
```

```

)
nx.draw_networkx_edges(
    G, pos, edgelist=[("4", "4")], connectionstyle="arc3,rad=3.0"
)

plt.axis("off")
plt.show()

```



Regular Graph:

A regular graph is a type of undirected graph where every vertex has the same number of edges and therefore degrees.

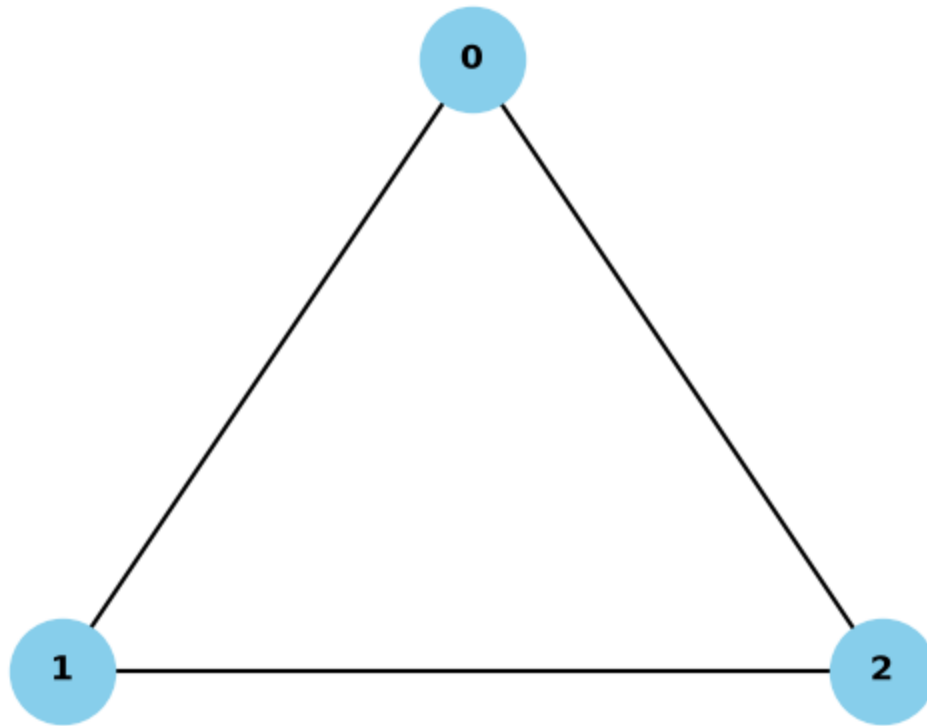
```

In [ ]: import matplotlib.pyplot as plt
import networkx as nx

# 3-vertex triangle (2-regular graph)
G = nx.cycle_graph(3)
pos = {0: (0, 1), 1: (-0.866, -0.5), 2: (0.866, -0.5)}

nx.draw_networkx(
    G, pos, node_color="#87CEEB", node_size=1400, font_weight="bold", width=1.5
)
plt.axis("off")
plt.show()

```



7.7: Based on Density

The density of a graph is calculated by comparing the number of edges to the number of edges. You can compute edge density by dividing the number of edges in the current graph by the maximum possible number of edges in a complete graph.

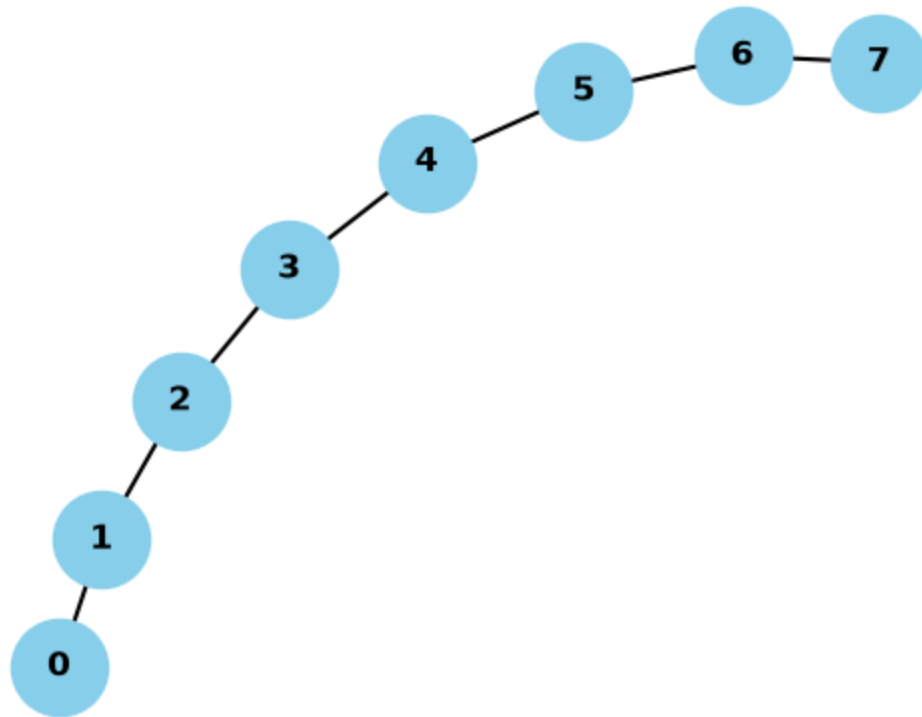
Sparse Graphs:

A sparse graph is a graph with relatively few edges compared to the number of vertices.

```
In [ ]: import matplotlib.pyplot as plt
import networkx as nx

# Sparse graph: 8 vertices and only 7 edges (out of 28 possible)
G = nx.path_graph(8)

nx.draw_networkx(
    G, node_color="#87CEEB", node_size=1200, font_weight="bold", width=1.5
)
plt.axis("off")
plt.show()
```



Dense Graphs:

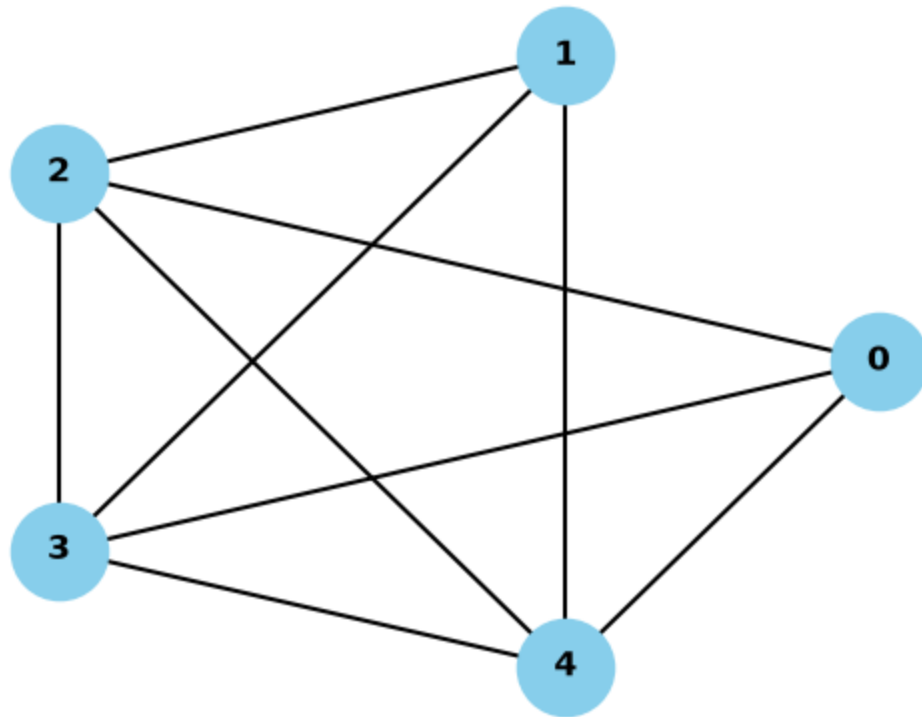
A dense graph is a graph with relatively more edges compared to the number of vertices.

```
In [ ]: import matplotlib.pyplot as plt
import networkx as nx

# Create a dense graph: K_5 missing only 1 edge (5 vertices, 9 edges)
G = nx.complete_graph(5)
G.remove_edge(0, 1)

pos = nx.circular_layout(G)

nx.draw_networkx(
    G, pos, node_color="#87CEEB", node_size=1200, font_weight="bold", width=1.5
)
plt.axis("off")
plt.show()
```



7.8: Based on Cycles

The cycles in a graph are essentially the number of loops that can be identified within the graph. More formally, a cycle is a closed path in which the starting and ending vertices are the same. There are three types of cycle graphs: cyclic, acyclic, and trees.

Cyclic Graph:

A cyclic graph contains at least one cycle

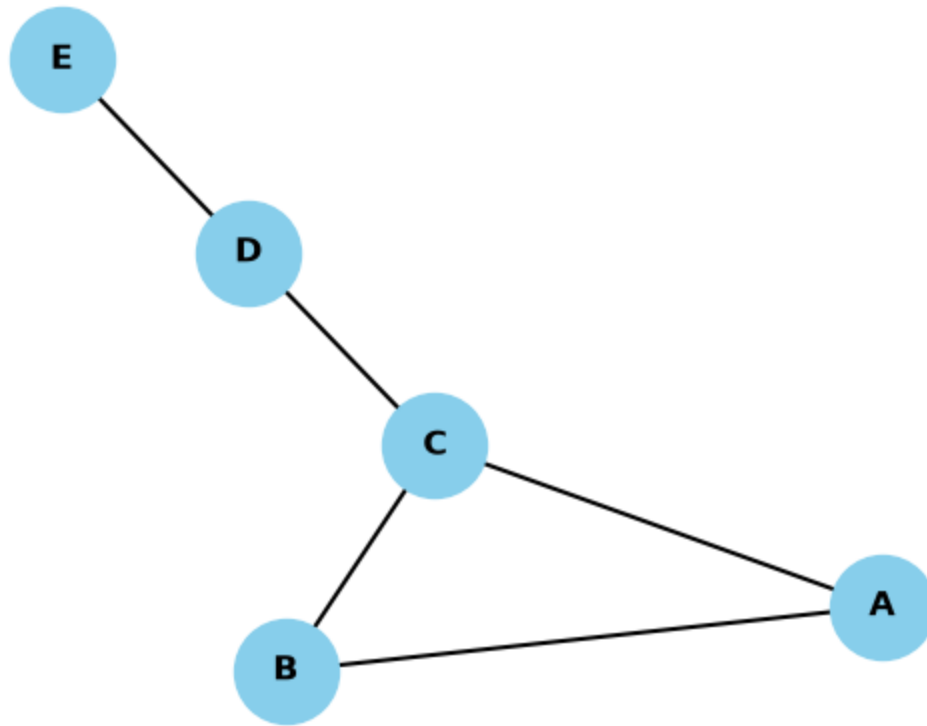
```

In [ ]: import matplotlib.pyplot as plt
import networkx as nx

# Define edges: triangle cycle (A, B, C) + extended path (C, D, E)
edges = [("A", "B"), ("B", "C"), ("C", "A"), ("C", "D"), ("D", "E")]
G = nx.Graph(edges)

pos = {
    "E": (-1.0, 1.2),
    "D": (-0.5, 0.6),
    "C": (0.0, 0.0),
    "B": (-0.4, -0.7),
    "A": (1.2, -0.5),
}

nx.draw_networkx(
    G, pos, node_color="#87CEEB", node_size=1400, font_weight="bold", width=1.5
)
plt.axis("off")
plt.show()
  
```



Acyclic Graph:

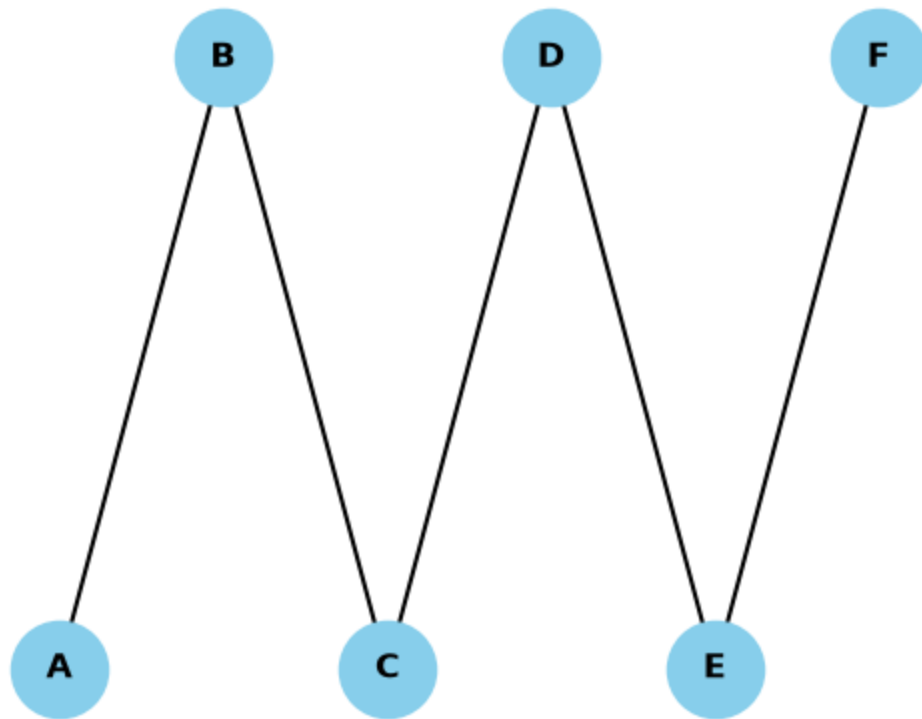
An acyclic graph is a graph with no cycles, which can be connected or disconnected.

```
In [ ]: import matplotlib.pyplot as plt
import networkx as nx

# Acyclic graph (tree path with turns, no closed loops)
edges = [("A", "B"), ("B", "C"), ("C", "D"), ("D", "E"), ("E", "F")]
G = nx.Graph(edges)

# Coordinates creating a zig-zag / turned path
pos = {
    "A": (0, 0),
    "B": (1, 1),
    "C": (2, 0),
    "D": (3, 1),
    "E": (4, 0),
    "F": (5, 1),
}

nx.draw_networkx(
    G, pos, node_color="#87CEEB", node_size=1200, font_weight="bold", width=1.5
)
plt.axis("off")
plt.show()
```



Trees:

A tree graph is a connected graph that contains no cycles (loops), where any two vertices are connected by exactly one simple path. A specific type of tree graph is a binary tree graph, which has a root node and every node has two left and right children nodes.

```
In [ ]: import matplotlib.pyplot as plt
import networkx as nx

# Define edges for binary tree
edges = [
    ("V1", "V2"),
    ("V1", "V3"),
    ("V2", "V4"),
    ("V2", "V5"),
    ("V3", "V6"),
    ("V3", "V7"),
]

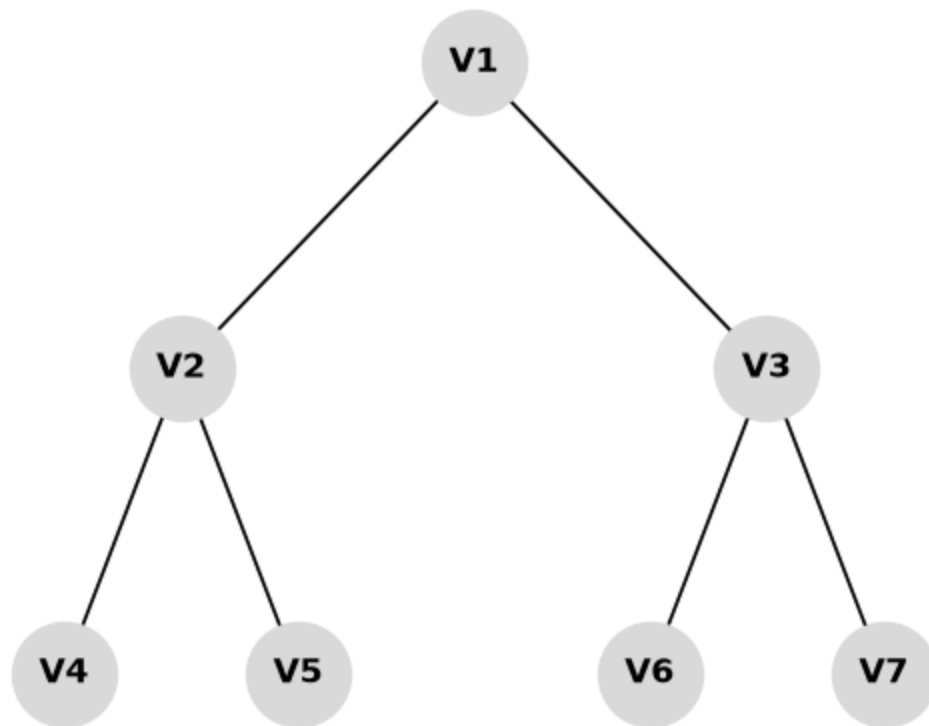
G = nx.Graph(edges)

# Hierarchical positions matching the image
pos = {
    "V1": (0.0, 2.0),
    "V2": (-1.0, 1.0),
    "V3": (1.0, 1.0),
    "V4": (-1.4, 0.0),
    "V5": (-0.6, 0.0),
    "V6": (0.6, 0.0),
    "V7": (1.4, 0.0),
}
```

```

nx.draw_networkx(
    G,
    pos,
    node_color="#D9D9D9",
    node_size=1400,
    font_weight="bold",
    width=1.2,
)
plt.axis("off")
plt.show()

```



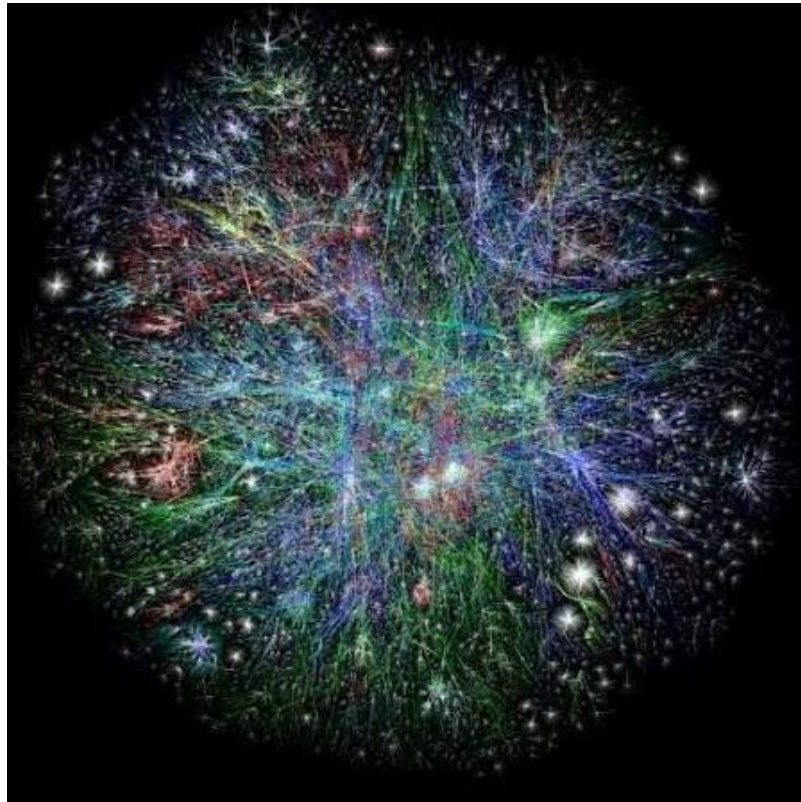
8: Usage of Graphs in Representing Real-World Relationships

There are many aspects of systems that are of interest to scientists, two being the nature of individual components and the nature of connections and interactions. However, many neglect a third aspect to these interacting systems, which is the pattern of connections between components. When analyzing the pattern of connections between components, scientists often use a graph network, where the components of the system are the network vertices and the connections are written as edges. There are many particular patterns which each graph network can have, heavily influencing the behavior of the system. The main advantage to using a graph network to model real-world relationships are because of their versatility and their abstract structure which only captures the connect patterns and very little else.

Since graph theory works in their abstract form, they can almost be applied to almost any system to be represented as a network, some better than others. Here are some notable examples of how graph theories can be applied to real-world relationships.

8.1: The Internet

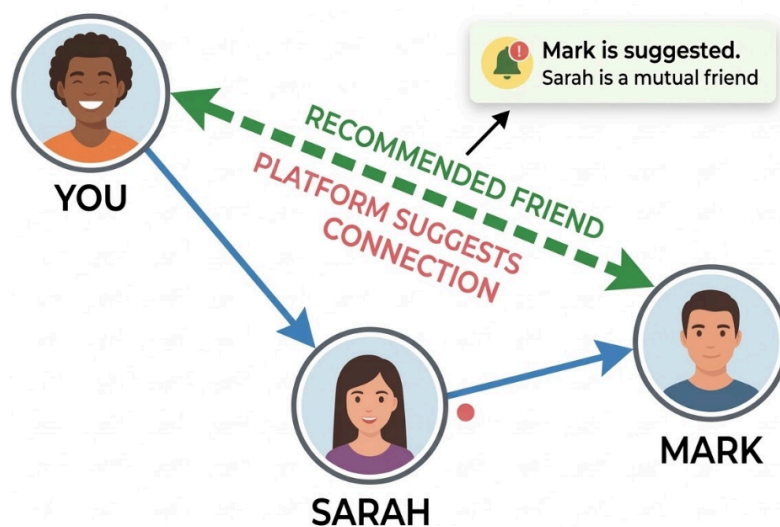
The internet can be modeled using graph theory by defining the computers/networks as vertices and the physical data connections as edges.



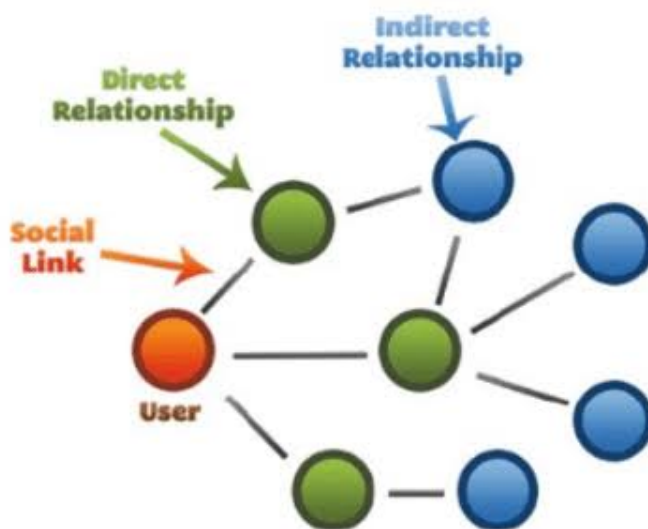
A Graphical Network Representation of the Internet

8.2: Social Networks

Social networks can be naturally modeled using graph theory by representing individuals as vertices and their relationships as edges. A prominent real-world application of this model is social media algorithms, which analyze edge patterns to identify mutual friends (vertices connected to the same neighboring vertices).



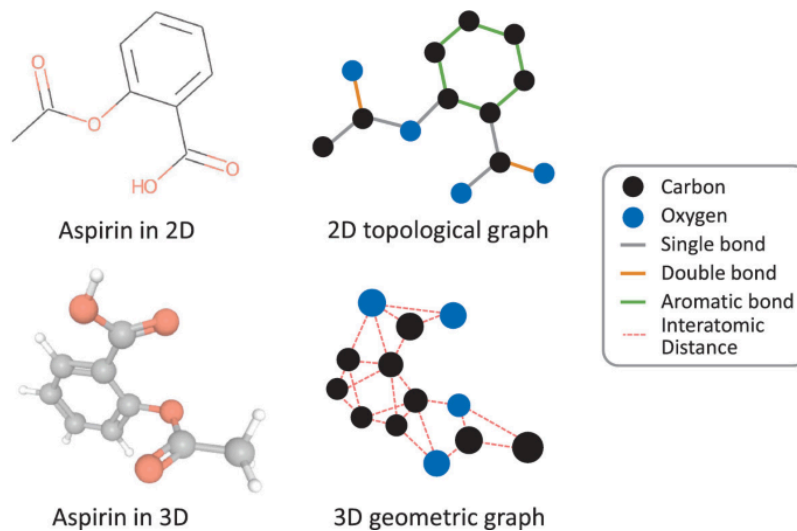
A Model of how Mutual Friends are Calculated using Graph Theory



Direct and Indirect Relationships in Social Networks

8.3: Chemical Molecules

Chemical molecules can also be modeled using graph theory by defining the elements as the vertices and their types of bonds as edges. In the example below, aspirin is being modeled in 2D and 3D graph representation.



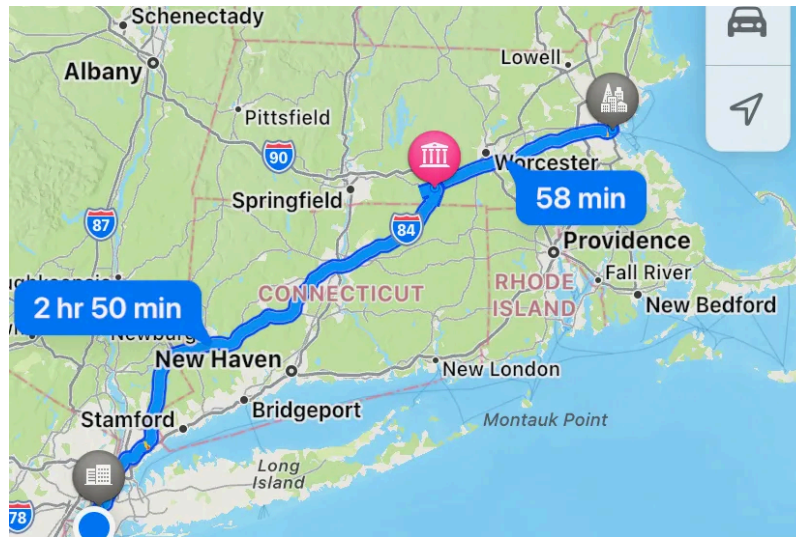
Aspirin Modeled in 2D and 3D using Graph Theory

8.4 Road/Transportation Structures

Roads can be modeled using graph theory by representing places as the vertices and the roads/routes as edges. One well-known algorithm used as a foundational algorithm in maps and routes is Dijkstra's Algorithm, which finds the shortest path from one vertex to all other vertices in a graph. We will dive deeper into Dijkstra's Algorithm in a later portion of the paper.



Navigation Maps Modeled using Graph Theory



Global Positioning System (GPS) in Apple Maps to Calculate Shortest Distance

In summary, networks are thus a powerful way to represent the PATTERNS of connections or interactions between parts of a system.

Part Two: STRONGLY REGULAR GRAPHS

1: Defining a Strongly Regular Graph

- **Combinatorial Definition:**
 1. for integer k , has k number of neighbors
 2. for integer λ as common neighbors, if vertices x & y share an edge, they also share the same number of mutual neighbors, vertices z
 3. for vertices x & y which are not neighbors (that don't share an edge) have the same number μ of vertices z as neighbors
- **Spectral Definition:**
 - A strongly regular graph is a finite regular graph with exactly 2 restricted eigenvalues
- These two definitions are proven to be equal by the A^2 identity, which uses the parameters k, λ, μ , where A is the Adjacency matrix of a Strongly Regular Graph with the eigenvalue k
- The A^2 identity is therefore able to prove that A has 2 eigenvalues using the parameter k, λ, μ
 - $A^2 = \lambda A + \mu(J - I - A) + kI = (\lambda - \mu)A + \mu J + (k - \mu)I$

Strongly regular graphs are defined by Brouwer and Van Maldeghem as a "finite regular graph with precisely two restricted eigenvalues." (Brouwer & Van Maldeghem, 2022) The graph X is a set of elements called vertices, where each pair of vertices may share a two-way, symmetric relationship called *adjacency*, such that no vertex is adjacent to itself. The pair of vertices xy is an *edge* where y is the *neighbor* of x .

The adjacency matrix A represents the connections of graph X . The table representation of this matrix has one row and one column for each vertex, with each cell comparing a pair of vertices. If x and y are vertices and xy is an edge, then $A_{xy} = 1$. If x and y are vertices and xy is not an edge, then $A_{xy} = 0$.

Consider the adjacency matrix A acting as a transformation. Inputting a vector into this matrix may output a new vector pointing in a different direction. For special vectors v , multiplying by A does not change the direction of v , instead it only stretches or shrinks this vector by some factor. v is an eigenvector, and the stretching factor is its eigenvalue, often represented by θ .

The number of specific eigenvectors which share the same eigenvalue is the eigenvalue's *multiplicity* and the complete list of a matrix's eigenvalues with their multiplicities is called the *spectrum*. The graph X is of *regular degree* (also known as *valency*) for some integer k , when every vertex has precisely k neighbors.

If every vertex has the same degree k , then the all-1s vector (a vector with each component equal to 1) will always be scaled exactly by k . This vector will be an eigenvector with eigenvalue k if and only if X is regular and of degree k (Brouwer & Van Maldeghem, 2022). With this in mind, the multiplicity of the eigenvalue k will be the number of connected components of X (Brouwer & Van Maldeghem, 2022). Importantly, for the definition of a strongly regular graph, an eigenvalue θ of this regular graph is referred to as *restricted* if it produces an eigenvector orthogonal to the all-1 vector (Brouwer & Van Maldeghem, 2022). Two vectors $u = [u_1, u_2, u_3, u_4]$ and $w = [w_1, w_2, w_3, w_4]$ are orthogonal exactly when their dot product is 0. $u \cdot w = (u_1 \times w_1) + (u_2 \times w_2) + (u_3 \times w_3) + (u_4 \times w_4) = 0$

With all of this in mind, a finite regular graph without restricted eigenvalues has at most one vertex, while a finite regular graph with only one restricted eigenvalue is *complete* or *edgeless* (Brouwer & Van Maldeghem, 2022). Circling back to Brouwer and Van Maldeghem's definition, "A *strongly regular graph* is a finite regular graph with precisely two restricted eigenvalues" (Brouwer & Van Maldeghem, 2022). This means that the graph not only has a fixed degree, but also reflects the fact that every pair of adjacent vertices shares another fixed number of common neighbors everywhere in the graph - regardless of its size.

```
In [4]: import networkx as nx
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# "Pentagon Graph" (C_5) -> smallest strongly regular graph SRG(5,2,0,1)
# v = 5 (vertices), k = 2 (degree), lambda = 0 (adjacent vertices with common neigh
G = nx.cycle_graph(5)
A = nx.to_numpy_array(G, dtype=int)
df = pd.DataFrame(A, index=G.nodes(), columns=G.nodes())
```

```

# create a figure with 1 row and 2 columns of plots (side by side)
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

# draw the graph G into the LEFT plot (axes[0])
# with_labels=True shows vertex numbers, node_color/node_size control appearance
nx.draw(G, with_labels=True, node_color='lightblue', node_size=800, font_size=14, ax=
axes[0].set_title("Pentagon Graph (C5)") # add a title above the left plot

# turn off the x/y axis lines and numbers for the RIGHT plot
axes[1].axis('off')

# build a table inside the right plot using our adjacency matrix DataFrame (df)
# cellText = the actual 0/1 values, rowLabels/collabels = the vertex numbers along
table = axes[1].table(cellText=df.values, rowLabels=df.index, colLabels=df.columns,
table.scale(1, 1.5) # stretch the table slightly taller so it's easier to read
axes[1].set_title("Adjacency Matrix") # add a title above the right plot

# arrange everything neatly so titles/labels don't overlap
plt.tight_layout()

# actually display the finished figure
plt.show()

# --STEP 1: Check if regular--
# determining if graph is strongly regular
degrees = [deg for node, deg in G.degree()] # G.degree() -> (vertex, degree) *pull i
print("Step 1: Degree of each vertex:", degrees) # print degree of every vertex
# set(degrees) removes duplicates from list of degrees -> if one unique value every
print("Is every vertex the same degree (regular)?", len(set(degrees)) == 1) # len i
k = degrees[0] # all degrees are equal, so first one can be k
print(f"Degree k = {k}") # print k

#--STEP 2: Check common neighbors for each adjacent pair (all should equal lambda)-
print("\nStep 2: Common neighbors for each adjacent pair:")
# G.edges() returns every edge as pair of vertices (x,y)
for x, y in G.edges():
    common = len(list(nx.common_neighbors(G, x, y)))
    print(f" Vertices {x} and {y} (adjacent): {common} common neighbors")

#--STEP 3: Check common neighbors for every non-adjacent pair (all should equal mu)
print("\nStep 2: Common neighbors for each non-adjacent pair:")
nodes = list(G.nodes()) # get plain list of all vertex labels
# loop through every unique vertex pair (i,j) without repeating pairs or comparing
for i in range(len(nodes)):
    for j in range(i + 1, len(nodes)):
        x, y = nodes[i], nodes[j]
        if not G.has_edge(x, y): # only check pairs not connected by an edge
            common = len(list(nx.common_neighbors(G, x, y)))
            print(f" Vertices {x} and {y} (non-adjacent): {common} common neighbors")

#--STEP 4: Conclusion based on combinatorial process--
print("\nStep 4: Since every adjacent pair shares the same number of common neighbo
print("and every non-adjacent pair shares the same number of common neighbors (mu),
print("this graph satisfies the strongly regular graph condition: SRG(v, k, lambda,

#--STEP 5: Verify using eigenvalue process--

```

```

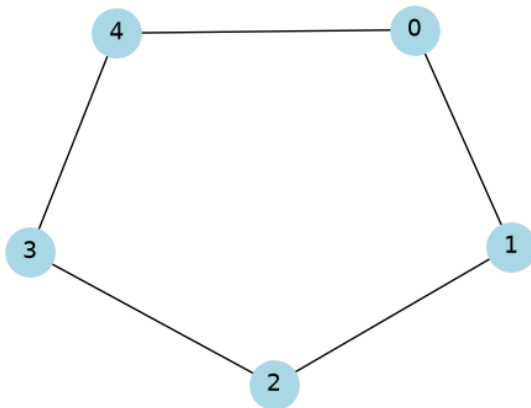
# compute eigenvalues of adjacency matrix A
eigenvalues, eigenvectors = np.linalg.eig(A)
eigenvalues_rounded = np.round(eigenvalues, 2) # round eigenvalues

print("Step 5: Eigenvalues of adjacency matrix:")
print(eigenvalues_rounded)

# count how many distinct eigenvalues there are
distinct_eigenvalues = set(eigenvalues_rounded)
print(f"\nNumber of distinct eigenvalues: {len(distinct_eigenvalues)}")
print("One of these is k (the degree). The remaining distinct values are the 'restricted eigenvalues'")
print("A strongly regular graph should have exactly 2 restricted eigenvalues (3 distinct eigenvalues total)")

```

Pentagon Graph (C5)



Adjacency Matrix

	0	1	2	3	4
0	0	1	0	0	1
1	1	0	1	0	0
2	0	1	0	1	0
3	0	0	1	0	1
4	1	0	0	1	0

Step 1: Degree of each vertex: [2, 2, 2, 2, 2]
Is every vertex the same degree (regular)? True
Degree $k = 2$

Step 2: Common neighbors for each adjacent pair:
Vertices 0 and 1 (adjacent): 0 common neighbors
Vertices 0 and 4 (adjacent): 0 common neighbors
Vertices 1 and 2 (adjacent): 0 common neighbors
Vertices 2 and 3 (adjacent): 0 common neighbors
Vertices 3 and 4 (adjacent): 0 common neighbors

Step 2: Common neighbors for each non-adjacent pair:
Vertices 0 and 2 (non-adjacent): 1 common neighbors
Vertices 0 and 3 (non-adjacent): 1 common neighbors
Vertices 1 and 3 (non-adjacent): 1 common neighbors
Vertices 1 and 4 (non-adjacent): 1 common neighbors
Vertices 2 and 4 (non-adjacent): 1 common neighbors

Step 4: Since every adjacent pair shares the same number of common neighbors (λ),
and every non-adjacent pair shares the same number of common neighbors (μ),
this graph satisfies the strongly regular graph condition: $\text{SRG}(v, k, \lambda, \mu)$.
Step 5: Eigenvalues of adjacency matrix:
[-1.62 0.62 2. -1.62 0.62]

Number of distinct eigenvalues: 3
One of these is k (the degree). The remaining distinct values are the 'restricted' eigenvalues.
A strongly regular graph should have exactly 2 restricted eigenvalues (3 distinct total, including k).

2: Importance of Strongly Regular Graphs in Combinatorics, Abstract Algebra, and Coding Theory

2.1: SRG in Combinatorics

Strongly Regular Graphs are a fundamental aspect of study within combinatorics, a type of mathematics which deals with counting, arranging, and organizing objects, without the use of calculus or continuous math. Many well known strongly regular graphs are derived from combinatorial constructions like Latin Square Graphs.

- Latin Squares are an $n \times n$ grid where each number spans between 1 and n , with absolutely no repeating numbers in a row or column. Latin Square Graphs are then constructed from Latin Squares, where each cell in the grid becomes a vertex (also known as nodes) and the number of nodes an LSG has is found using n^2 . Two nodes share an edge if they are in the same column, row, or if they are simply the same number regardless of which row or column.
- To find k , the number of neighbors a cell has in Latin Square graphs, $k = 3(n - 1)$. This is because in order for a cell to actually share an edge it must meet exactly one of the three requirements (row, column, or number). Since a cell is neighbors with every other

cell in its row, it has $(n - 1)$ neighbors in its row. It also becomes neighbors with every other cell in its column so another $(n - 1)$. Finally, it is also neighbors with the same number that is repeated in every other column or row so another $(n - 1)$. This totals to $k = (n - 1) + (n - 1) + (n - 1)$, which is simplified to $k = 3(n - 1)$.

- Every pair of vertices that shares an edge has n common neighbors, therefore $\lambda = n$ common neighbors. This is the case as the pair of nodes would share $(n-2)$ common neighbors with the rest of the row. Then, the first number in the pair that would be repeated in the column of the second number would be the next common neighbor (+1), and vice versa where the second number which is repeated in the column of the first number would be the other common neighbor (+1). Therefore, $\lambda = (n - 2) + 1 + 1 = n$
- Meanwhile, when looking at nodes that don't share an edge, they will always share a constant value of common neighbors as $\mu = 6$. This constant value of 6 does not depend on the size of the grid and instead stems from the criteria being met which are row, column, and number. This is because when you have 2 cells that are not neighbors, the first cell has 2 common neighbors from the cell that also shares a row with the second cell as well as the number from cell 2 being repeated in the column of cell 1. Then the second cell has 2 common neighbors from the cell that shares a column with the first cell and the number of the first cell being repeated in the column of cell 2. Lastly, there are two more common neighbors from the same number in cell 1 and 2 being repeated in the other columns. In this way μ will always be six regardless of the size of the Latin Square Graph.

Latin Square Graphs satisfy all the requirements in the definition of Strongly Regular Graphs, as each node has the same k number of neighbors, each pair has the same λ common neighbors when they share an edge, and has the same μ common neighbors if they don't share an edge regardless of which cells are chosen.

```
In [ ]: import networkx as nx
import numpy as np
import matplotlib.pyplot as plt

# First Create Initial Matrix Outline
n = 3
mat = []
node_rlist = []
node_clist = []
for i in range(n):
    mat.append([0] * n)
#print(*mat, sep="\n")

# Start to insert values
for r in range(n):
    for c in range(n):
        mat[r][c] = ((r+c) % n)+1
        node_rlist.append(r+1)
```

```

        node_clist.append(c+1)

node_list = list(zip(node_rlist, node_clist))
print("Latin Square with %dx%d grid\n" % (n,n))
print(*mat, sep="\n")
print("\n")
print("Nodes of the Graph: \n", node_list)
print("\n")

LSG = nx.Graph()
LSG.add_nodes_from(node_list)

# Create 2 different tuples of the same node list to compare and make sure that the
pc = 0
for i in range(len(node_list)):
    for j in range(i+1, len(node_list)):
        node1 = node_list[i]
        node2 = node_list[j]
        pc+=1

# check for same row, column or number which are conditions to identify neighbors (
        if node1[0] == node2[0] or node1[1] == node2[1] or mat[node1[0]-1][node1[1]]
            # print("connected")
            LSG.add_edge(node1, node2)
        #print(f" pair count: {pc}, node1: {node1} & node2: {node2} \n")

print("The number of edges the Latin Square Graph has: ", LSG.number_of_edges())
print("\n")
print("The degrees or number of neighbors the Latin Square Graph has: \n", dict(LSG

# a new loop to check for common and uncommon neighbors
lambda_list = set() #set of common neighbors
mu_list = set() # set of uncommon neighbors

pc = 0
for i in range(len(node_list)):
    for j in range(i+1, len(node_list)):
        node1 = node_list[i]
        node2 = node_list[j]
        pc+=1

        common_neighbors = set(LSG[node1]) & set(LSG[node2])
        count = len(common_neighbors)

        if LSG.has_edge(node1, node2):
            lambda_list.add(count)
        else:
            mu_list.add(count)

print(f"calculated value:  $\lambda$  = {lambda_list}, predicted value:  $\lambda$  = n = {n}")
print(f"calculated value:  $\mu$  = {mu_list}, predicted  $\mu$  = 6")

```

```

#draw graph
pos = nx.spring_layout(LSG, seed=42)
plt.figure(figsize=(6,6))
nx.draw(LSG, pos, with_labels=True, node_color='lightblue', node_size=600, font_size=12)
plt.title(f"Latin Square Graph (n={n})")
plt.show()

```

Latin Square with 3x3 grid

```

[1, 2, 3]
[2, 3, 1]
[3, 1, 2]

```

Nodes of the Graph:

```

[(1, 1), (1, 2), (1, 3), (2, 1), (2, 2), (2, 3), (3, 1), (3, 2), (3, 3)]

```

The number of edges the Latin Square Graph has: 27

The degrees or number of neighbors the Latin Square Graph has:

```

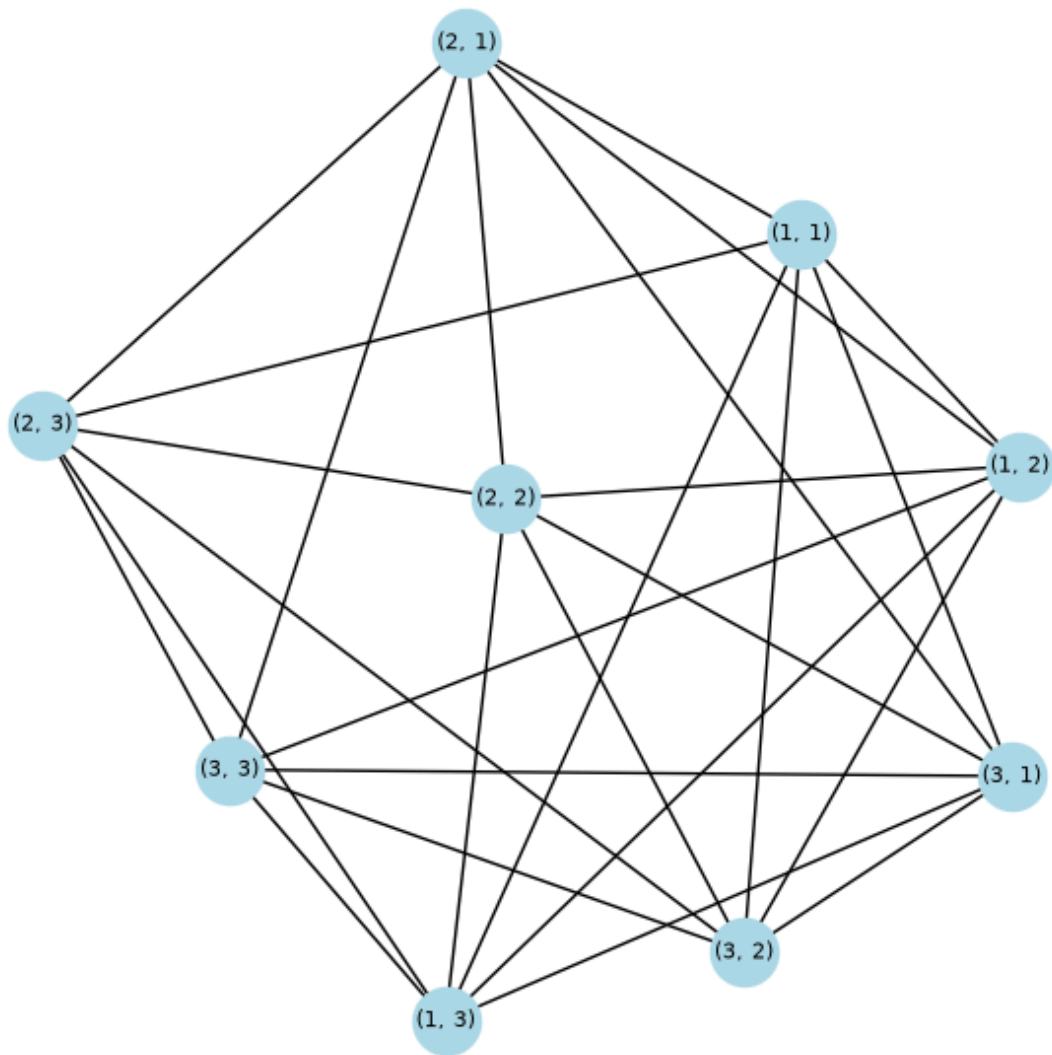
{(1, 1): 6, (1, 2): 6, (1, 3): 6, (2, 1): 6, (2, 2): 6, (2, 3): 6, (3, 1): 6, (3, 2): 6, (3, 3): 6}

```

calculated value: $\lambda = \{3\}$, predicted value: $\lambda = n = 3$

calculated value: $\mu = \{6\}$, predicted $\mu = 6$

Latin Square Graph ($n=3$)



2.2: Strongly Regular Graphs in Abstract Algebra

The use of strongly regular graphs in abstract algebra is primarily seen in group theory, which is also known as the study of symmetry. In comparison to proving how Strongly Regular graphs are prevalent in combinatorics where the criteria was proven through hand counting, in group theory λ , and μ remain constant through the use of pure symmetry.

- In abstract algebra, a permutation group is a group with G , paired with a function that reorganizes the elements of a set X . Where every part of G plays a role in reorganizing the pieces of X .
- The group G is transitive if any part of X can be mapped to any other. This means any element can be reached starting from any other element in the group.
- A group is of rank 3 when G has 3 orbits, where the pairs split into D , E and F , where D is $x=y$. Meanwhile if E and F are symmetric then they have 2 different graphs, (X, E) which

refers to an adjacent or edge, and other one which is (X, F) which refers to the complement or non-adjacent.

- Automorphism is where vertices are relabeled or reorganized to ensure that the adjacency relationship is preserved, this causes every edge and non-edge to be shuffled without disturbing their characteristics.
- Since G is transitive, for adjacent pairs, it allows for any adjacent pair of vertices to be mapped out by automorphisms to any other adjacent pair. In this way automorphism can retain the exact graph structure, while simultaneously preserving the number of common neighbors. This causes every adjacent pair to share the same constant λ , and similarly every non-adjacent pair must share the same constant μ .

Therefore, the amount of common neighbors of 2 vertices relies only on whether they are equal, adjacent, or nonadjacent, and does not depend on the chosen vertices.

2.3: Strongly Regular Graphs in Coding Theory

In coding theory Strongly Regular Graphs mainly show up in the Hoffman Bound. This bound creates a strict upper limit on the size of a clique, which is a set of vertices that are all mutually connected to one another, in a strongly regular graph ($|D| \leq 1 + k/(-s)$) and is based on the graph's degree k and it's smallest eigenvalue s . In other words, the subset of a strongly regular graph can never go over the size limit found based on the graph's spectrum, regardless of how the graph is otherwise structured.

```
In [ ]: # build petersen graph
c = nx.petersen_graph()
a = nx.to_numpy_array(c) # turn graph into numpy array in order to list the eigenvalues
eigenvalues = np.linalg.eigvalsh(a)
print(f"Eigenvalues: {eigenvalues}\n")

#find k the largest value
k = np.max(eigenvalues)
print(f"largest eigenvalue k: {k}\n")

# find s the smallest eigenvalue
s = np.min(eigenvalues)
print(f"smallest eigenvalue s: {s}\n")

#calculate Hoffman Bound
D = 1+(k/-s)
print(f"Hoffman Bound on clique size: {D}\n")

# find actual max clique size
b = list(nx.find_cliques(c))
print(f"Maximal Cliques found: {b}\n")

largest_size = max(len(clique) for clique in b)
print(f"Actual largest clique size: {largest_size}")
```

Eigenvalues: [-2. -2. -2. -2. 1. 1. 1. 1. 1. 3.]

largest eigenvalue k: 3.0

smallest eigenvalue s: -2.0000000000000001

Hoffman Bound on clique size: 2.4999999999999999

Maximal Cliques found: [[0, 1], [0, 4], [0, 5], [2, 1], [2, 3], [2, 7], [3, 8], [3, 4], [6, 8], [6, 1], [6, 9], [7, 9], [7, 5], [8, 5], [9, 4]]

Actual largest clique size: 2

- Using the Petersen graph to apply the Hoffman Bound, it is found that $k = 3$ and $s \approx -2$, leading to a bound of $|D| \leq 2.5$, but the size of a clique must be a whole number, which entails that the cliques of the Petersen graph must have no more than 2 vertices. This is found to be true as the actual size of the largest clique has exactly 2 vertices, which illustrates our theoretical prediction, showing that the eigenvalue rule belonging to strongly regular graphs restricts the structure that can exist in it.

3: Famous Strongly Regular Graphs

3.1: Petersen Graph

The Petersen Graph was built by a Danish mathematician named Julius Petersen. He specifically constructed this graph to refute a mathematical conjecture that was widely known. It claimed that any connected, bridgeless cubic graph could always be edge colored with just 3 colors. So Petersen went out of his way to create a graph that fulfilled all the conditions, it was connected, bridgeless and cubic, but because of its specific structure it could not be edge-colored with 3 colors. The Petersen graph is also the unique strongly regular graph with parameters $(v, k, \lambda, \mu) = (10, 3, 0, 1)$, where no other graph has the same set of parameters. It is also a complement of the triangular graph $T(5)$ which happens to be a specific case of another well known Strongly Regular graph, the Johnson graph.

```
In [ ]: g = nx.petersen_graph() #Petersen Graph
l = nx.line_graph(g) #turns above petersen graph into a Line graph

print(f"Edges in Petersen Graph: {g.number_of_edges()}\n")
print(f"Nodes in Line Graph: {l.number_of_nodes()}\n")

lc = nx.coloring.greedy_color(l)
#print(lc)

colors = set(lc.values())
print(f"distinct colors: {len(colors)}\n")

edgelist = list(lc.keys())
for i in range(len(edgelist)):
    for j in range(i+1, len(edgelist)):
        edge1 = edgelist[i]
```

```

edge2 = edgelist[j]
if edge1[0] in edge2 or edge1[1] in edge2:
    if lc[edge1] == lc[edge2]:
        print(f"Conflict between {edge1} & {edge2}")

edge_colors = []
for u, v in g.edges():
    edge_colors.append(lc.get((u,v)))

pos = nx.shell_layout(g, nlist=[list(range(5)), list(range(5, 10))])
nx.draw(g, pos, with_labels=True, edge_color=edge_colors, node_color='lightblue')
plt.title(f"Edge-Colored Petersen Graph")

```

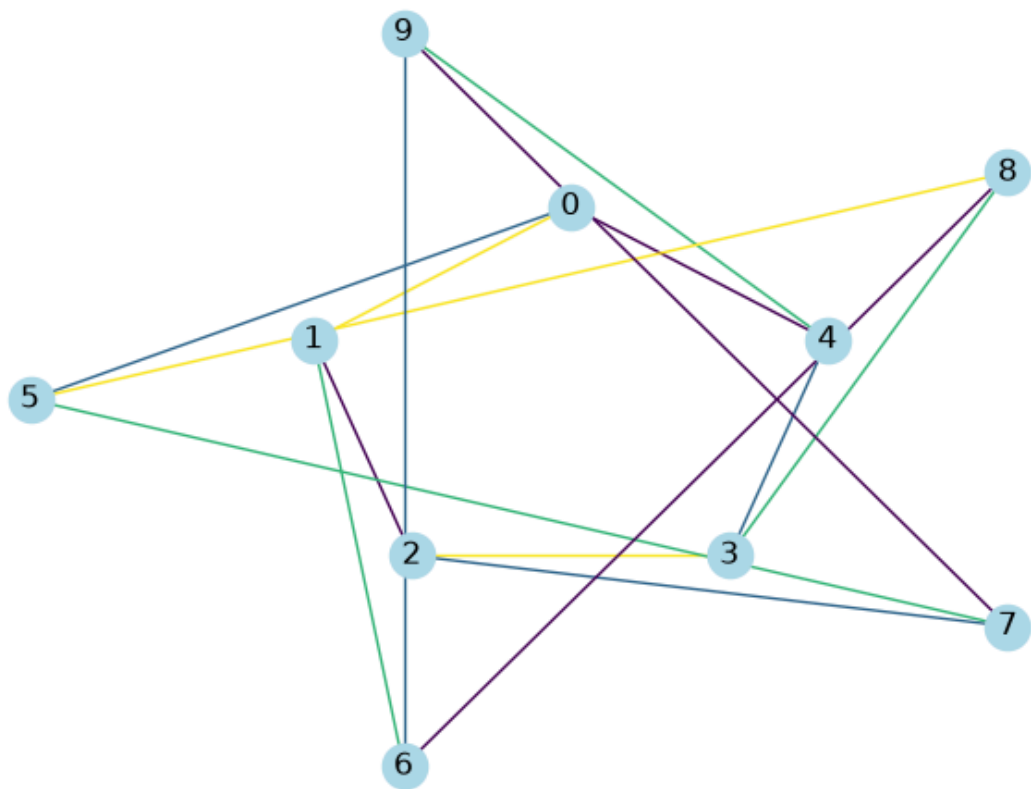
Edges in Petersen Graph: 15

Nodes in Line Graph: 15

distinct colors: 4

Out[]: Text(0.5, 1.0, 'Edge-Colored Petersen Graph')

Edge-Colored Petersen Graph



- To go through this historical conjecture, the edge coloring method was followed using a greedy algorithm, by turning a Petersen graph into a Line graph, in order to compare the number of edges and nodes respectively. To confirm that Petersen graphs can't be

edge colored with 3 colors, the greedy algorithm required 4 colors for all 15 edges. Although the algorithm doesn't prove that using 3 colors is completely impossible, the outcome is sufficient enough to align with established theory that the chromatic index is 4. This follows Julius Petersen's claim of 4 colors instead of 3, as a strong counterexample.

3.2: Johnson Graph

Johnson graphs, as previously mentioned are another well known case of Strongly Regular Graphs. It is made up of a set Ω and an integer $d \geq 0$, making the Johnson graph $J(\Omega, d)$, and its vertices are all possible d -element subsets of Ω . If 2 subsets of a Johnson graph share $d - 1$ elements, between the two subsets, meaning that they now share an edge. When $d = 2$ it is a specific case of the Johnson graph known as the triangular graph $T(m)$. Meanwhile, the complement of the triangular graph $T(5)$ is the Petersen graph.

3.3: Hamming Graph

Similar to Johnson graphs, Hamming graphs are also composed of a set Ω and an integer $d \geq 0$, but unlike Johnson graphs, which use subsets, the Hamming graph uses tuples $H(d, \Omega)$. Here the vertices of a Hamming graph are all possible d -tuples of the set Ω , and if two of these tuples differ by exactly one position, then they are adjacent and this is known as Hamming distance 1. A special case of the Hamming graph occurs when $H(2, q)$ as this structure is relatively close to the build of Latin Square Graphs, without the same number rule. This specific $H(2, q)$ is known as a Lattice graph $L_2(q)$ or the $q \times q$ grid, a concept which is very similar to the one introduced at the beginning.

Part Three: ADJACENCY MATRICES

This section explores how graphs can be represented with matrices, what information can be read from those matrices, how weighted graphs change the representation, and why adjacency matrices are unusually interesting for strongly regular graphs.

1: Adjacency Matrix Definition

A graph $G(V, E)$ can be translated into a numerical representation which we call an adjacency matrix. Each vertex corresponds to one row and one column of the matrix. For a simple graph, if two vertices i and j are connected by an edge then $A_{ij} = 1$, otherwise $A_{ij} = 0$. The adjacency matrix for a simple undirected graph is symmetric, as an edge connecting vertex i to vertex j also connects j to i . The diagonal entries are 0 as a vertex cannot be connected to itself in a simple undirected graph. (Lehman et al., 2010)

2: Translating a Graph to an Adjacency Matrix

In order to construct an adjacency matrix, choose an ordering for the vertices. Each row and column represents one vertex. Enter 1 for every pair that is connected by an edge and 0 otherwise. Since undirected edges work in both directions, the matrix contains matching entries across the main diagonal. (Lehman et al., 2010)

Here is an example with the Petersen Graph and its adjacency matrix.

```
In [ ]: import networkx as nx
import numpy as np
import matplotlib.pyplot as plt

# Make the Petersen graph and its adjacency matrix
G = nx.petersen_graph()
A = nx.to_numpy_array(G, dtype=int)

# Graph and Matrix Display
fig, axes = plt.subplots(1, 2, figsize=(11, 5))

nx.draw(G, with_labels=True, node_size=700, ax=axes[0])
axes[0].set_title("Petersen Graph")

axes[1].imshow(A)
axes[1].set_title("Adjacency Matrix")
axes[1].set_xlabel("Vertex")
axes[1].set_ylabel("Vertex")
axes[1].set_xticks(range(10))
axes[1].set_yticks(range(10))

plt.tight_layout()
plt.show()
```

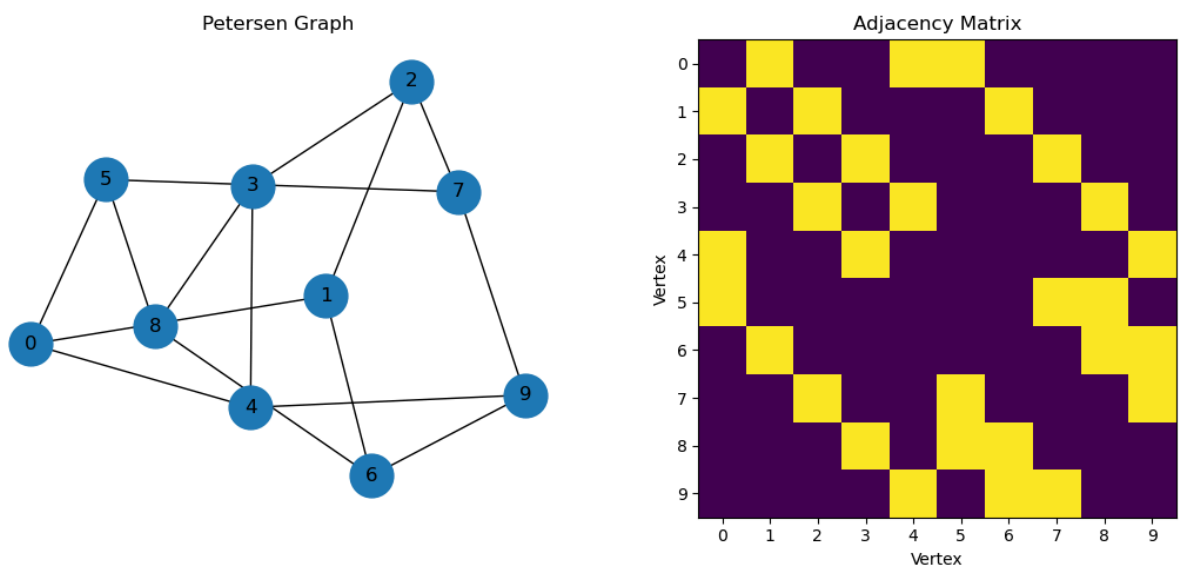


Figure 1. The Petersen graph and its adjacency matrix.

3: Insights from an Adjacency Matrix

The degree of a vertex is the number of entries equal to 1, since each 1 represents a neighbor of the vertex. Powers of the adjacency matrix contain additional information about movement through the graph. In particular, the entry $(A^2)_{ij}$ counts the number of two-step walks from vertex i to vertex j . For two different vertices, this also gives the number of common neighbors they share. (Lehman et al., 2010)

```
In [ ]: # degree of each vertex given by the sum of the row
degrees = A.sum(axis=1)

# A^2 contains the number of two step walks
A2 = A @ A

print("Degree of each vertex:")
print(degrees)

print("\nA squared:")
print(A2)
```

```
Degree of each vertex:
[3 3 3 3 3 3 3 3 3 3]
```

```
A squared:
[[3 0 1 1 0 0 1 1 1 1]
 [0 3 0 1 1 1 0 1 1 1]
 [1 0 3 0 1 1 1 0 1 1]
 [1 1 0 3 0 1 1 1 0 1]
 [0 1 1 0 3 1 1 1 1 0]
 [0 1 1 1 1 3 1 0 0 1]
 [1 0 1 1 1 1 3 1 0 0]
 [1 1 0 1 1 0 1 3 1 0]
 [1 1 1 0 1 0 0 1 3 1]
 [1 1 1 1 0 1 0 0 1 3]]
```

Considering the Petersen graph, every row sums to 3, meaning each vertex has degree 3. In A^2 , the diagonal entries are 3 as well. The off-diagonal entries show the number of common neighbors shared by pairs of different vertices.

4: Weighted Graphs and Adjacency Matrices

In a weighted graph, the entries of the adjacency matrix do not have to be only 0 or 1. Instead, A_{ij} can contain the numerical weight assigned to the edge between vertices i and j . Depending on the graph, this weight might represent distance, cost, capacity, or another quantity. (Lehman et al., 2010) The sum of the row then represents the weighted degree, or total strength, instead of only the number of neighbors.

5: Strongly Regular Graphs and Adjacency Matrices

A strongly regular graph has parameters (v, k, λ, μ) with v vertices where each vertex has a degree of k , every pair of adjacent vertices has λ common neighbors, and each pair of non-

adjacent vertices has μ common neighbors. Because the entries of A^2 count two-step walks, they also count common neighbors, so the strongly regular condition can be expressed as

$$A^2 = kI + \lambda A + \mu(J - I - A).$$

I is the identity matrix and J is the matrix whose entries are all 1. This equation expresses the common neighbor structure of a strongly regular graph as a single matrix identity. (Spielman, 2009)

5.1: Petersen Graph

The Petersen graph is an SRG $(10, 3, 0, 1)$. The graph has 10 vertices, where each vertex has a degree of 3, adjacent vertices have 0 common neighbors, and non-adjacent vertices have 1 common neighbor. If we substitute these values into the general identity, we get

$$A^2 = 2I - A + J.$$

```
In [ ]: # Verifying the strongly regular graph identity for the Petersen graph
n = A.shape[0]
I = np.eye(n, dtype=int)
J = np.ones((n, n), dtype=int)

left_side = A @ A
right_side = 2 * I - A + J

print("Does A^2 = 2I - A + J?")
print(np.array_equal(left_side, right_side))
```

Does $A^2 = 2I - A + J$?
True

The output `True` verifies that the Petersen graph satisfies the adjacency-matrix identity associated with its strongly regular graph parameters.

5.2: Cycle Graph C_5

The cycle graph C_5 is an SRG with parameters $(5, 2, 0, 1)$. (Spielman, 2009) Each vertex has a degree of 2, adjacent vertices have no common neighbors, and non-adjacent vertices have exactly one common neighbor. Its matrix identity is

$$A^2 = I - A + J.$$

```
In [ ]: # C5 and verifying its strongly regular graph identity
C5 = nx.cycle_graph(5)
B = nx.to_numpy_array(C5, dtype=int)

I5 = np.eye(5, dtype=int)
J5 = np.ones((5, 5), dtype=int)

print("C5 adjacency matrix:")
```

```

print(B)

print("\nDegree of each vertex:")
print(B.sum(axis=1))

print("\nDoes  $B^2 = I - B + J$ ?")
print(np.array_equal(B @ B, I5 - B + J5))

```

C_5 adjacency matrix:

```

[[0 1 0 0 1]
 [1 0 1 0 0]
 [0 1 0 1 0]
 [0 0 1 0 1]
 [1 0 0 1 0]]

```

Degree of each vertex:

```
[2 2 2 2 2]
```

Does $B^2 = I - B + J$?

```
True
```

The C_5 computation gives a degree of 2 for each vertex and returns True for its SRG matrix identity. Both the C_5 and the Petersen graph show the same type of strongly regular matrix structure.

References

Ajorlou, A. (2018). Lecture 2: Introduction to graph theory [Lecture notes]. MIT OpenCourseWare. https://ocw.mit.edu/courses/1-022-introduction-to-network-models-fall-2018/resources/mit1_022f18_lec2/

Aspnes, J. (n.d.). Graph theory [Course notes]. Yale University. <https://www.cs.yale.edu/homes/aspnes/pinewiki/attachments/GraphTheory/graph-theory.pdf>

Bazett, T. [Dr. Trefor Bazett]. (2017, July 8). Reflexive, symmetric, and transitive relations on a set [Video]. YouTube. https://www.youtube.com/watch?v=q0xN_N7I_Kw

Brouwer, A. E., & Van Maldeghem, H. (2022). Strongly regular graphs. Cambridge University Press. <https://homepages.cwi.nl/~aeb/math/srg/rk3/srgw.pdf>

GeeksforGeeks. (2023, May 31). Check if a graph is strongly, unilaterally or weakly connected. <https://www.geeksforgeeks.org/dsa/check-if-a-graph-is-strongly-unilaterally-or-weakly-connected/>

GeeksforGeeks. (2024, January 10). Types of graphs in graph theory with examples. <https://www.geeksforgeeks.org/dsa/graph-types-and-applications/>

GeeksforGeeks. (2024, May 13). Mathematics | Graph theory basics - Set 2. <https://www.geeksforgeeks.org/engineering-mathematics/mathematics-graph-theory->

basics/

Information System on Graph Classes and Their Inclusions. (n.d.). Graphclass: reflexive.
https://www.graphclasses.org/classes/gc_581.html

Lehman, E., Leighton, F. T., & Meyer, A. R. (2010). Graph theory. In Mathematics for computer science (chap. 5). MIT OpenCourseWare. https://ocw.mit.edu/courses/6-042j-mathematics-for-computer-science-fall-2010/f471f7b7034fabe8bbba5507df7d307f/MIT6_042JF10_chap05.pdf

Nykamp, D. Q. (n.d.). An introduction to networks. Math Insight.
https://mathinsight.org/network_introduction

Sedgewick, R., & Wayne, K. (2011). Undirected graphs. In Algorithms (4th ed., sec. 4.1). Addison-Wesley. <https://algs4.cs.princeton.edu/41graph/>

Spielman, D. A. (2009). Strongly regular graphs, part 1 (Lecture 23) [Lecture notes]. Yale University. <https://www.cs.yale.edu/homes/spielman/561/2009/lect23-09.pdf>

TrevTutor. (2015, May 10). Introduction to graph theory - discrete mathematics [Video]. YouTube. <https://www.youtube.com/watch?v=HkNdNpKUByM>

Tutorialspoint. (n.d.). Graph theory - types of graphs.
https://www.tutorialspoint.com/graph_theory/graph_theory_types_of_graphs.htm

W3Schools. (n.d.). DSA Dijkstra's algorithm.
https://www.w3schools.com/dsa/dsa_algo_graphs_dijkstra.php